

Project: [INF2004] 4-Legged Spider Robot (Servos)

Team Name: CS13

Name	SIT Number
Mohammad Adli Jumahat	2401516
Swathi Ravichandran	2402695
Lai Yueyin Shyann	2402456
Subhashini Rajkumar	2402769
Borse Srushti Ganesh	2401618

Project: [INF2004] 4-Legged Spider Robot (Servos).....	1
1. Project Overview.....	3
2. System Context & Assumptions.....	3
3. Functional Requirements (FR).....	5
4. Non-Functional Requirements (NFR).....	7
5. High-Level Architecture & Design.....	8
Overall Architecture Diagram.....	8
Spider Servo State Machine.....	9
Spider Algorithm Statechart Diagram.....	11
Sequence Diagrams.....	13
Use Case Diagram.....	19
6. Driver Interfaces.....	19
Driver Module 1: Heat Sensor Driver.....	19
Driver Module 2: Motor Driver.....	21
Bench Setup.....	22
Known Limitations.....	23
Driver Module 3: Radar Driver.....	23
Driver Module 4: Microphone Driver.....	25
7. Detailed Design.....	28
8. Test Plan & Results.....	48
9. Traceability (Week 9 & Week 13).....	56
10. Appendices (Week 13) - in case you have additional info you want to add on.....	60
11. Sign-off (Week 13).....	60
References:	61

1. Project Overview

The Spider Robot project focuses on developing a semi-autonomous agent for post-disaster search and rescue in collapsed structures. Its primary function is to survey damage and detect survivors in areas too dangerous for human rescuers.

The robot is designed for sophisticated operation through integrated sensing and navigation:

- **Survivor Detection:** The robot uses a microphone to detect specific frequency distress signals and then employs a heat sensor at close range to confirm human presence.
- **Intelligent Navigation:** The system uses a radar module to detect obstacles and assess the traversability of paths (including height). The robot follows a mapping algorithm to explore, stop upon obstacle detection, turn to find a new route, and intelligently reroute by backtracking to previous decision points when paths are exhausted. The direction of movement is prioritized by triaging for detected sound cues.
- **Operational Role:** The robot communicates findings via LoRa modules to Rescue Operators, who provide necessary human oversight and interpret the data, as the system is not intended for fully autonomous decision-making. The entire system is powered by a portable power bank.

2. System Context & Assumptions

System Context:

We focus on the development of a spider-inspired robotic system designed for post-disaster environments, specifically in the aftermath of earthquakes or building collapses. The primary goal of the system is to aid search and rescue operations by entering unstable and confined areas that are too dangerous or inaccessible for human rescuers.

The spider robot functions as a semi-autonomous agent capable of navigating through rubble, detecting obstacles, identifying survivors, and relaying information to rescue personnel. It integrates multiple sensors: a radar module for obstacle detection and path assessment, a heat sensor to confirm the presence of humans, and a microphone to detect sounds in specific frequency ranges that may indicate survivors calling for help. Communication between the robot and rescue operators is facilitated through LoRa modules, allowing for data transmission and remote control in environments where traditional wireless communication may be limited. The system is powered by a portable power bank to ensure mobility and independence from external power sources.

The system exists within a broader rescue framework where human operators deploy, monitor, and interpret the robot's findings. Survivors interact with the system indirectly, through heat and sound cues, while the environment itself (debris, structural instability, and confined spaces) influences how effectively the spider can operate. In this way, the spider robot forms part of a larger ecosystem of tools and personnel dedicated to post-disaster response.

Assumptions:

We have made several assumptions regarding this spider robot and regarding the environment, technology and operational context:

1. **Environmental Assumptions:**
 - The collapsed structure provides at least some navigable spaces large enough for the robots to crawl through.
 - The terrain, while irregular, is not completely impassable, and the robot's locomotion system can manage rubble and uneven surfaces.

- Background noise and temperature variations do not completely mask human-related cues (sound or heat).
2. Sensor Assumptions:
- The single mmWave radar sensor is sufficient for detecting obstacles in front of the robot but does not provide full 360° coverage.
 - The microphone can detect the presence of sound (such as a shout or call for help) but cannot determine the direction or exact location of the source.
 - The heat sensor can only confirm a human presence when in very close proximity or direct contact with the heat source.
 - Sensor readings may require operator interpretation or cross-verification rather than full autonomous classification.
3. Operational Assumptions:
- Rescue operators are trained to monitor sensor outputs and manually adjust the robot's behavior when necessary.
 - Survivors will produce detectable cues (sound or body heat) that can trigger the robot's sensors when nearby.
 - The robot's semi-autonomous system depends on human oversight for decision-making and path adjustment.
 - The robot is not expected to function entirely autonomously; human oversight and decision-making remain essential throughout its operation.

3. Functional Requirements (FR)

FR ID	Title	Requirement	Rationale	Verification Method	Verification Statement
FR-001	Standby Pose	The spider robot shall be able to default to its standby pose upon powering on	Having the ability to be able to stand by itself upon powering on without any human assistance is convenient	Test	The spider stands with the command sent from the pico
FR-002	Basic Movement	The spider robot shall be able to move in forward, backward, left and right direction using its multi-legged servo system.	Having the ability to move around in multiple directions is essential for navigating through debris-filled areas.	Test	The spider moves with the command sent from the pico
FR-003	Basic Turning	The spider robot shall be able to rotate clockwise and anti-clockwise direction using its multi-legged servo system.	Having the ability to rotate around is essential for looking around its surroundings in debris-filled areas.	Test	The spider turns with the command sent from the pico
FR-004	Heat Detection	The spider robot shall be able to detect human heat signature (between 36°C to 40°C) using the heat sensor within 10 cm radius ± 5 cm (This is the maximum range of the sensor)	Heat detection can help to identify survivors in its surroundings.	Test	The spider robot returns human temperature between 36°C to 40°C
FR-005	Sound Detection	The spider robot shall be able to capture and analyze audio frequencies between 300-3000Hz to detect human distress calls and screams using the microphone within 1 meter radius ± 10 cm.	Sound detection can help to identify survivors calling for help.	Test	The spider robot receives, categorises and analyzes radio frequencies to detect human distress calls and screams

		(This is the maximum range of the sensor)			
FR-006	Object Detection	The spider robot shall be able to detect nearby objects using the mm wave radar within 0.2 meters radius. Threshold is from 20 cm to 500 cm	Being able to detect objects such as debris can help the robot decide where it can manoeuvre to in debris-filled areas.	Test	The mm wave sensor on the robot is able to identify objects placed at set distance (20cm, 50cm, 100cm)
FR-007	Multi-Sensor Confirmation	The spider robot shall be able to confirm nearby survivors using its heat and sound detection Within a 10cm radius.	By cross-referencing both Sound and Heat Detection, it can help reduces false positives and improves survivor detection reliability	Test	The spider robot is able to confirm nearby survivors using its heat and sound sensors within a 10cm radius.
FR-008	Path Routing	The spider robot shall be able to navigate to potential survivors by combining Heat, Sound and Object Detection	By cross-referencing Heat, Sound and Object Detection, it can help the spider robot to move towards potential survivors while navigating in debris-filled areas.	Test	The spider robot is able to determine possible human survivors using a combination of temperature sensor input - 36°C to 40°C to confirm humans, microphone between 300-3000Hz to detect human in distress and object detection to identify human.
FR-009	Path Finding	The spider robot shall be able to navigate to different areas to find potential survivors by combining the Object Detection and its Path Finding Algorithm	By receiving data from Object Detection and inserting it to the Path Finding Algorithm, the spider will be able to move to new areas to help find more potential survivors	Test	The spider is able to have the algorithm make decisions to be able to navigate around the set path

4. Non-Functional Requirements (NFR)

NFR ID	Category	Description	Measurable Target	Acceptance Test
NFR-001	Reliability	Robot should operate continuously without the servos dying	90% powered up without overheating during 15 mins shift	Simulate the spider on standby to verify the uptime
NFR-002		System should continue to function if sensor fails	Core components running without certain sensors operating	Disable 1 sensor during test, verify data being sent
NFR-003	Performance	Detect and respond to nearby obstacle quickly	≤ 200 ms sensor-to-decision latency	Time obstacle detection to motor stop
NFR-004		Detect and report distress sounds promptly	≤ 5 s from sound capture to alert	Play audio samples and time response
NFR-005	Power	Operate on power bank for a minimum period	≥ 15 mins continuous runtime	Run until battery depletion
NFR-006		Power Bank easy to replace	≤ 1 min swap time	Time battery replacement process
NFR-007	Communication And Interoperability	Reliable long-range (Lora) communication	≥ 200 m in obstructed area	Field test LoRa range
NFR-008		Low delay in sending and receiving commands	Response time ≤ 1 s	Monitor transmission logs
NFR-009	Data handling	Store information even if signal is lost	Saves at least 10 mins of recent data	Disconnect and verify data recovery and availability
NFR-010	Sensor accuracy	Detect humans accurately	Heat sensor error within $\pm 3^{\circ}\text{C}$	Compare readings to thermal reference
NFR-011	Sensor accuracy	Obstacle detection precision	$\pm 10\text{cm}$ within 5m range	Control distance test

5. High-Level Architecture & Design

Overall Architecture Diagram

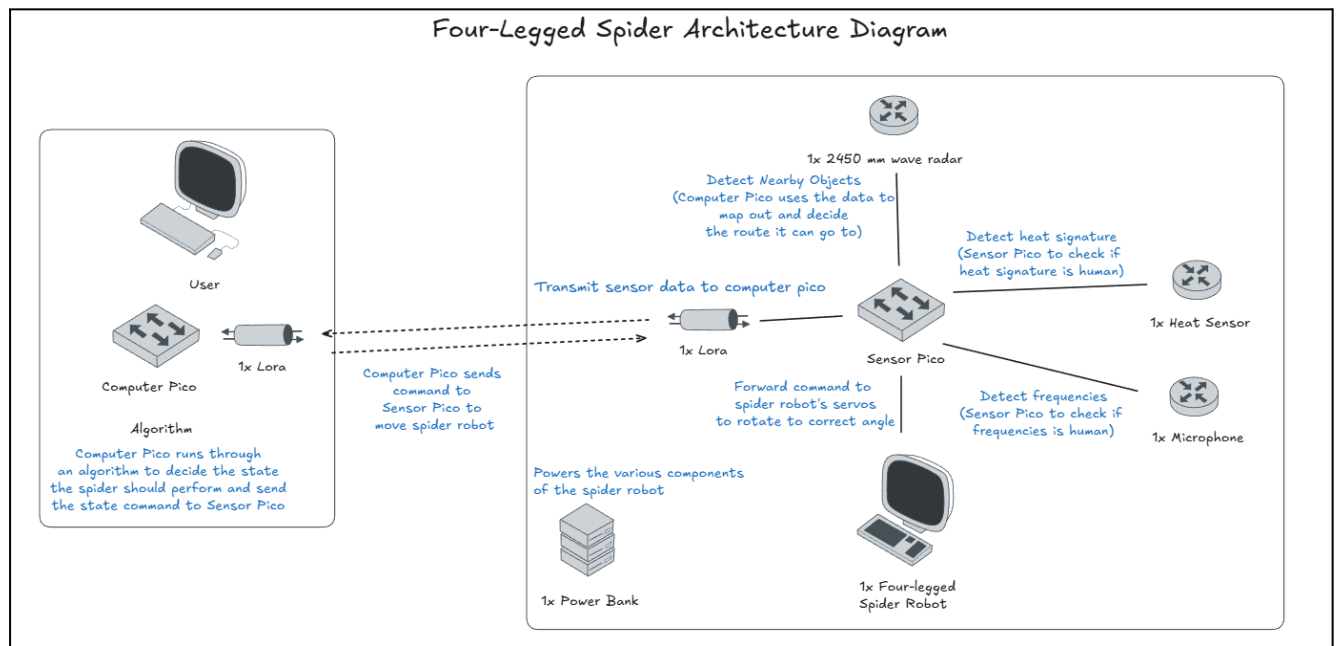


Figure 1. Four-Legged Spider Architecture Diagram

Design Element ID: D1.1 (Spider Servo System)

Description: The sensor pico controls the states of the servos. Through the use of a state machine, the servos hold their respective angles based on the current state of movement the spider is in.

Design Element ID: D1.2 (Computer Pico to Sensor Pico Lora Connection)

Description: The Sensor Pico and Computer Pico are connected through their respective lora components. The Sensor Pico sends the respective sensors' data to the Computer Pico. The Computer Pico sends state commands to the Sensor Pico.

Design Element ID: D1.3 (Spider Algorithm)

Description: The Computer Pico receives data from the Sensor Pico and runs through its logic algorithm to output the best course of action for the spider to perform. It will then send a state command back to the Sensor Pico for the spider robot to execute.

Design Element ID: D1.4 (MMWave Radar)

Description: The MMWave Radar scans and detects nearby objects in front of the spider robot. The Sensor Pico will then send the data to the Computer Pico if an object is detected.

Design Element ID: D1.5 (Heat Sensor)

Description: The Heat Sensor scans and detects nearby heat signatures in front of the spider robot. The Sensor Pico will then check if the temperature of the heat signatures are within range of human body temperature and flag out to the Computer Pico if so.

Design Element ID: D1.6 (Microphone Sensor)

Description: The Microphone Sensor scans and detects nearby audio frequencies around the spider robot. The Sensor Pico will then check if the frequency of the audios are within range of human shriek and flag out to the Computer Pico if so.

Design Element ID: D1.7 (Power Bank)

Description: The Power bank powers both the sensor pico and the robot spider servos through their respective USB cables so that the robot spider is able to perform its duties.

Spider Servo State Machine

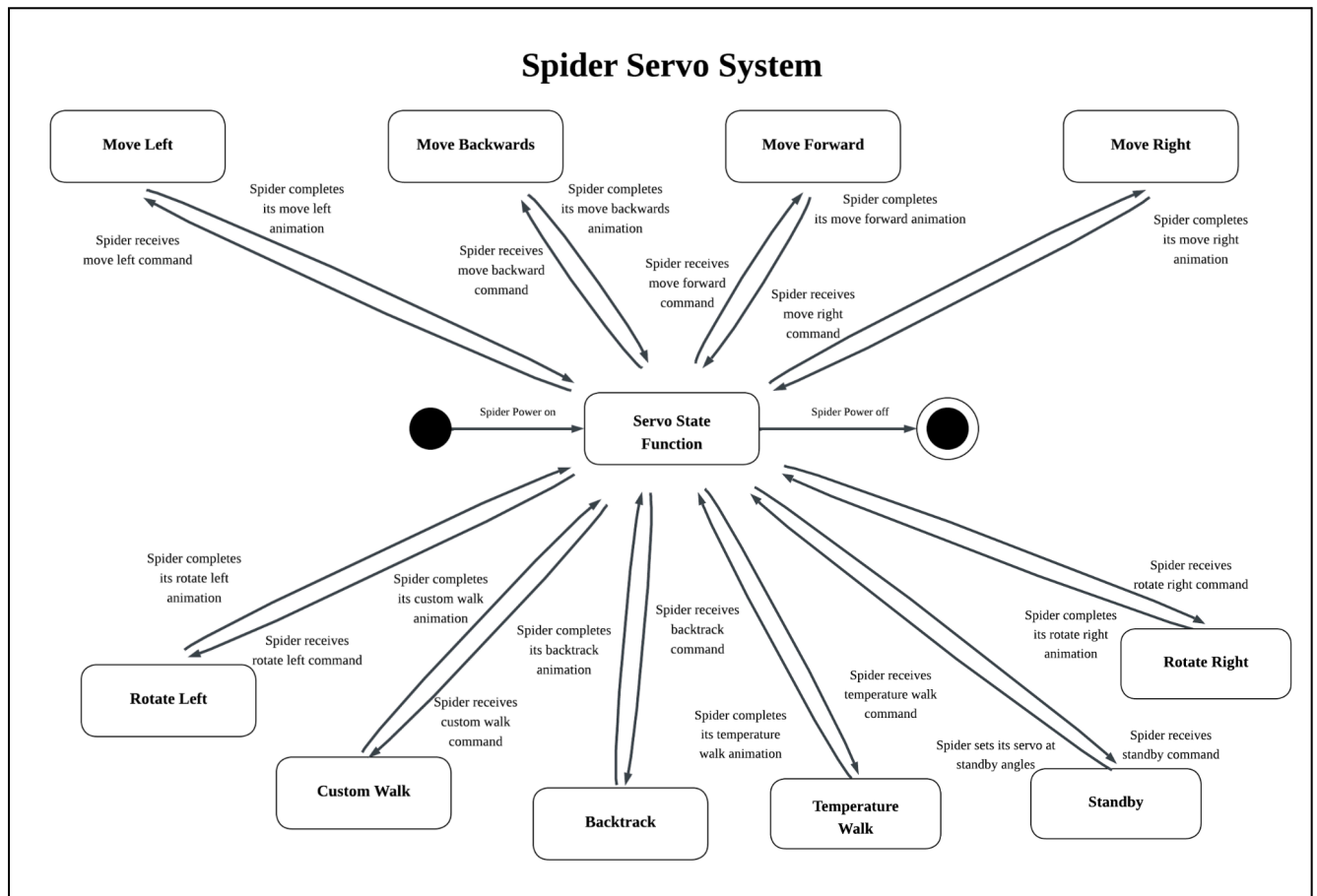


Figure 2. Spider Servo System

Design Element ID: D2.1 (Standby Mode)

Description: The default state of the spider upon powering on. The individual servos rotate and hold on to their respective angles to be in a standby stance. It will run SPIDER_STANDY state in the servo_leg_state function.

Design Element ID: D2.2 (Move Forward)

Description: Upon receiving command from the computer pico, the individual servos will rotate to their respective sequence of angles to perform a move forward animation. It will run SPIDER_MOVE_FORWARD state in the servo_leg_state function.

Design Element ID: D2.3 (Move Backward)

Description: Upon receiving command from the computer pico, the individual servos will rotate to their respective sequence of angles to perform a move backward animation. It will run SPIDER_MOVE_BACKWARD state in the servo_leg_state function.

Design Element ID: D2.4 (Move Left)

Description: Upon receiving command from the computer pico, the individual servos will rotate to their respective sequence of angles to perform a move left animation. It will run SPIDER_MOVE_LEFT state in the servo_leg_state function.

Design Element ID: D2.5 (Move Right)

Description: Upon receiving command from the computer pico, the individual servos will rotate to their respective sequence of angles to perform a move right animation. It will run SPIDER_MOVE_RIGHT state in the servo_leg_state function.

Design Element ID: D2.6 (Rotate Left)

Description: Upon receiving command from the computer pico, the individual servos will rotate to their respective sequence of angles to perform a rotate left animation. It will run SPIDER_ROTATE_LEFT state in the servo_leg_state function.

Design Element ID: D2.7 (Rotate Right)

Description: Upon receiving command from the computer pico, the individual servos will rotate to their respective sequence of angles to perform a rotate right animation. It will run SPIDER_ROTATE_RIGHT state in the servo_leg_state function.

Design Element ID: D2.8 (Custom Walk)

Description: Upon receiving command from the computer pico, the individual servos will rotate to their respective sequence of angles to perform a move forward animation, custom walk edition. This state is used for walking towards an object after the algorithm detects an object. It will run SPIDER_CUSTOM_WALK state in the servo_leg_state function.

Design Element ID: D2.9 (Temperature Walk)

Description: Upon receiving command from the computer pico, the individual servos will rotate to their respective sequence of angles to perform a move forward animation, temperature walk edition. This state is used for walking towards a potential human to scan their temperature. It will run SPIDER_TEMPERATURE_WALK state in the servo_leg_state_function.

Design Element ID: D2.10 (Backtrack)

Description: Upon receiving command from the computer pico, the individual servos will rotate to their respective sequence of angles to perform a sequence of animation. This state is for the spider robot to backtrack to the direction it comes from. It will run SPIDER_ROTATE_LEFT twice before SPIDER_MOVE_FORWARD in the servo_leg_state_function.

Spider Algorithm Statechart Diagram

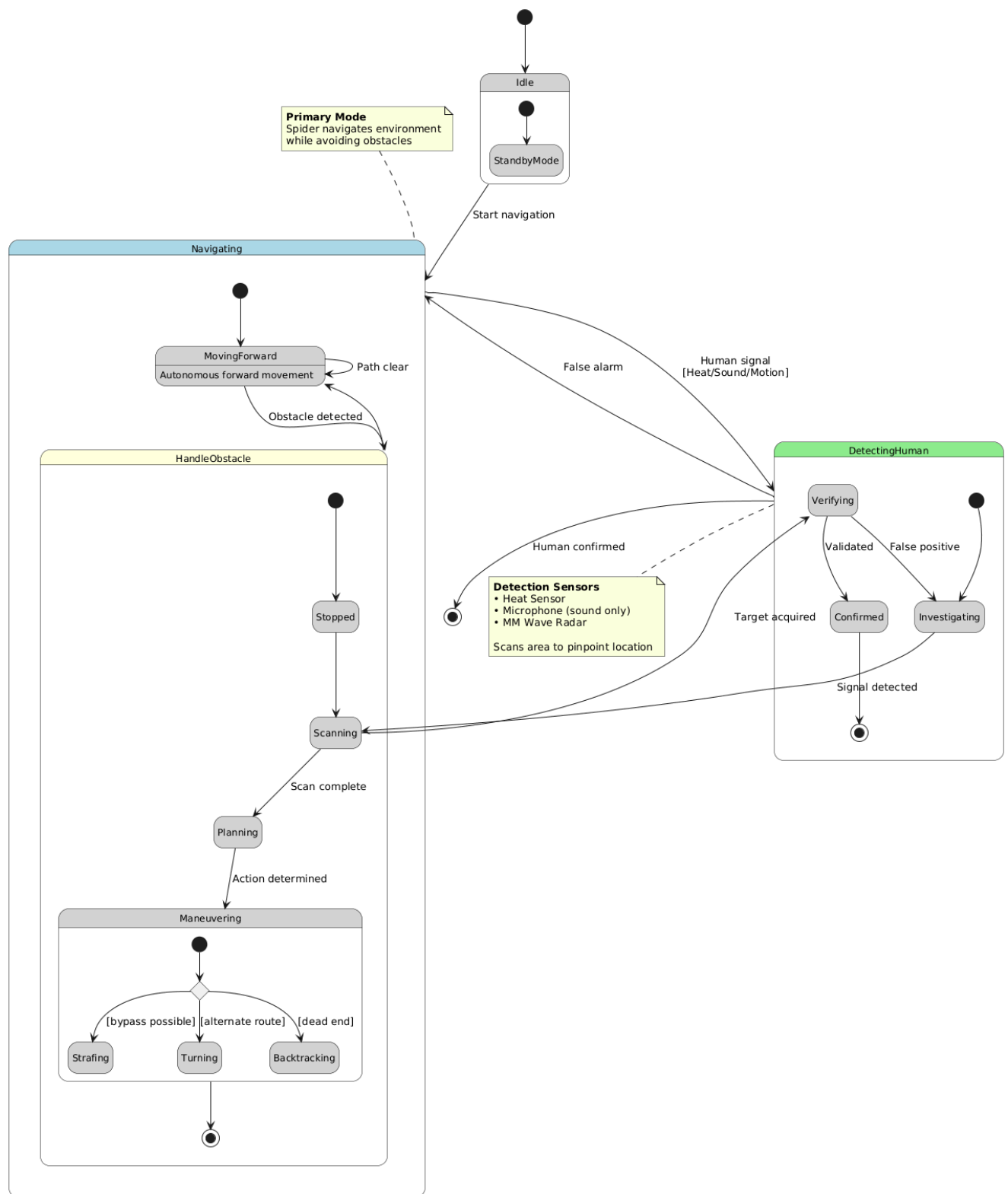


Figure 3. Spider Object Detection and Path Finding Algorithm

Design Element ID: D3.1 (Overall System Mode):

The system begins in the Idle state. Upon receiving a "Start navigation" command, it transitions to the StandbyMode, which acts as the initialization and ready state before entering the Navigating primary mode. The system constantly monitors for external stimuli (like a "Human signal") that can interrupt normal navigation.

Design Element ID: D3.2 (Navigating Mode - MovingForward):

This is the default operating state for environment exploration (Primary Mode). The robot continuously executes Autonomous forward movement until a sensor detects an obstacle in its direct path ("Obstacle detected"). If the environment remains clear, the robot continues to move forward ("Path clear" self-transition).

Design Element ID: D3.3 (HandleObstacle Mode - Scanning):

Upon detecting an obstacle, the robot immediately transitions to **Stop** and then to the **Scanning** state. In this state, the robot performs a localized scan (like a 360-degree rotation) to analyze the junction, log potential path vectors, and determine the nature of the blockage. The state ends when the scan is complete and the environment status is logged ("Scan complete").

Design Element ID: D3.4 (HandleObstacle Mode - Planning & Maneuvering):

After scanning, the system enters the Planning state to determine the appropriate response based on the scan results ("Action determined"). This leads to the Maneuvering state, which resolves the obstacle. It has three possible sub-outcomes:

- **Strafing** (Bypass possible, e.g., a small obstacle).
- **Turning** (Alternate route, e.g., selecting an unexplored path at a fork).
- **Backtracking** (Dead end, e.g., executing a backtrack maneuver).

Design Element ID: D3.5 (DetectingHuman Mode):

The system transitions into this high-priority mode when a human-like signal is detected (e.g., Heat/Sound/Motion). It consists of two sub-states:

- **Verifying**: The robot actively seeks confirmation that the signal is a human ("validated"). If not, it's logged as a "False positive" and the robot returns to the primary Navigating mode.
- **Investigating**: If the signal is confirmed ("signal detected"), the robot maneuvers to pinpoint the target's location ("Target acquired") until confirmation is achieved ("Confirmed"). A Human confirmed event leads to mission completion (or a terminal state).

Design Element ID: D3.6 (Transition: Human Confirmed):

Upon successfully confirming a human target, the robot immediately transitions from the DetectingHuman mode to a terminal state (represented by the black circle exit), effectively stopping navigation as the mission goal is achieved.

Sequence Diagrams

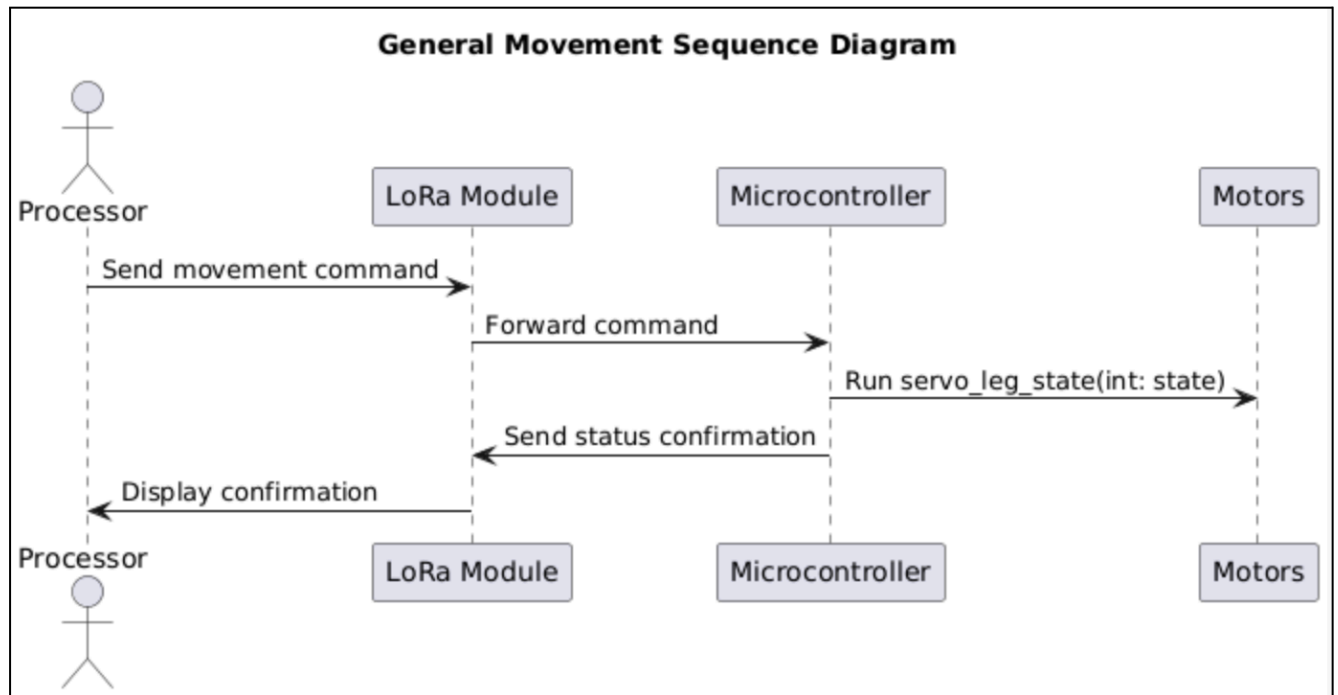


Figure 4. General Movement Sequence Diagram

Design Element ID: D4.1 (Processor)

Description: Its primary function is to define and send the high-level locomotion intent to the communication layer. The Processor's code must generate the standardized command string, and transmit/receive the message to the LoRa Module to conduct the movement sequence

Design Element ID: D4.2 (LoRa Module)

Description: It is responsible for forwarding commands from the processor to the micro controller and sending back status confirmation

Design Element ID: D4.3 (Microcontroller)

Description: Executes movement commands received from the LoRa module, to run servo control functions to translate into physical motor movements. It then generates status confirmation. The microcontroller is powered by the powerbank, connecting to the respective sensors and sensor drivers.

Design Element ID: D4.4 (Motors)

Description: Hardware actuators responsible for moving the robot's legs according to commands from the microcontroller.

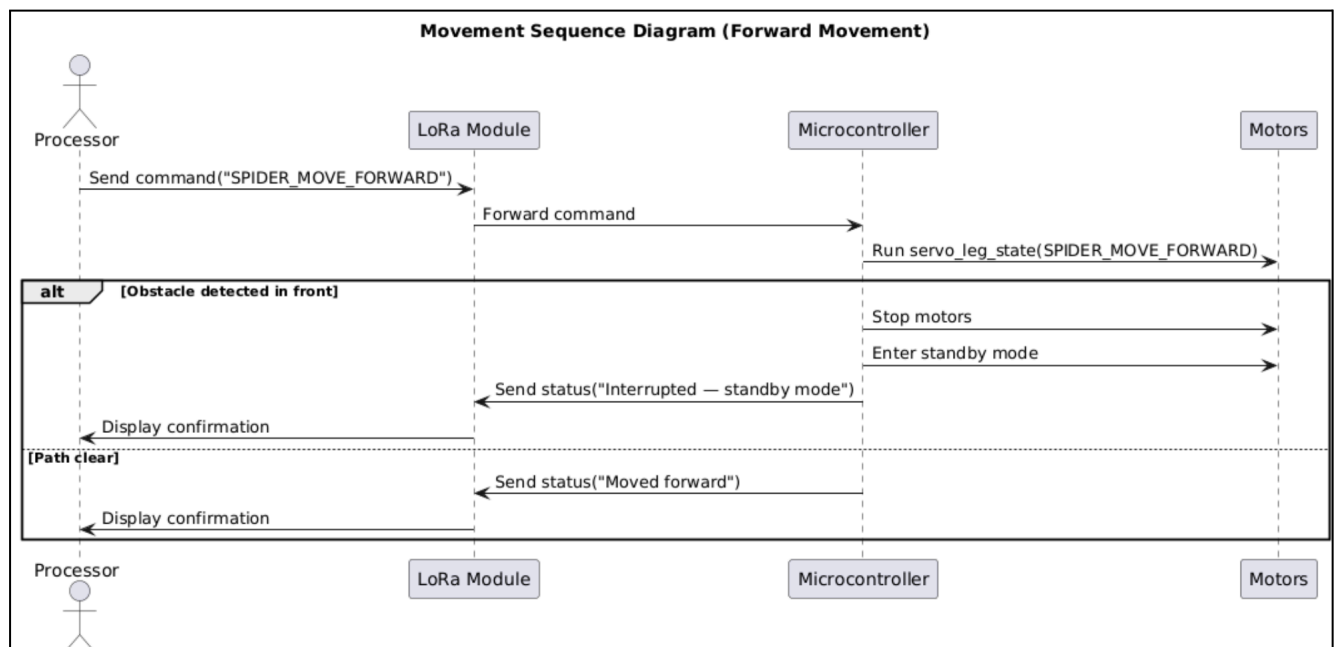


Figure 5. Forward Movement Sequence Diagram

The Spider Robot's "Move Forward" sequence starts with the Control System commanding the Movement Sequence Module to provide the necessary gait sequence and leg rotation instructions. The Control System sends these as PWM signals to the Motor Driver, which converts them into hardware signals to physically execute the leg movements. A continuous loop then repeats this sequence to maintain forward momentum until a stop command is received.

Design Element ID: D5.1 (Processor)

Description: Its primary function is to define and send the high-level locomotion intent to the communication layer. The Processor's code must generate the standardized command string, in this case, "SPIDER_MOVE_FORWARD", and transmit this message to the LoRa Module to begin the movement sequence

Design Element ID: D5.2 (LoRa Module)

Description: It is responsible for reliably receiving the high-level command from the Processor and relaying the "Forward command" to the Microcontroller for execution. It also handles the transmission of status messages—whether successful completion ("Moved forward") or an obstacle interruption ("Interrupted - standby mode")—back to the Processor for display and confirmation.

Design Element ID: D5.3 (Microcontroller)

Description: It receives the "Forward command" and runs the actual servo control function, specifically `run_servo_leg_state(SPIDER_MOVE_FORWARD)`, to translate the command into physical motor movements. The Microcontroller is also responsible for continuously monitoring the environment (via sensors like the MM Wave Radar) and initiating the stop sequence (stopping the motors and entering standby mode) if an obstacle is detected.

Design Element ID: D5.4 (Alternate Sequence)

Description: This design element requires the system's immediate, time-sensitive response to detecting an obstacle while moving forward. The sequence involves three immediate actions initiated by the Microcontroller: Stop motors, Enter standby mode, and sending the "Interrupted - standby mode" status message back to the Processor. This ensures the robot halts safely before any collision.

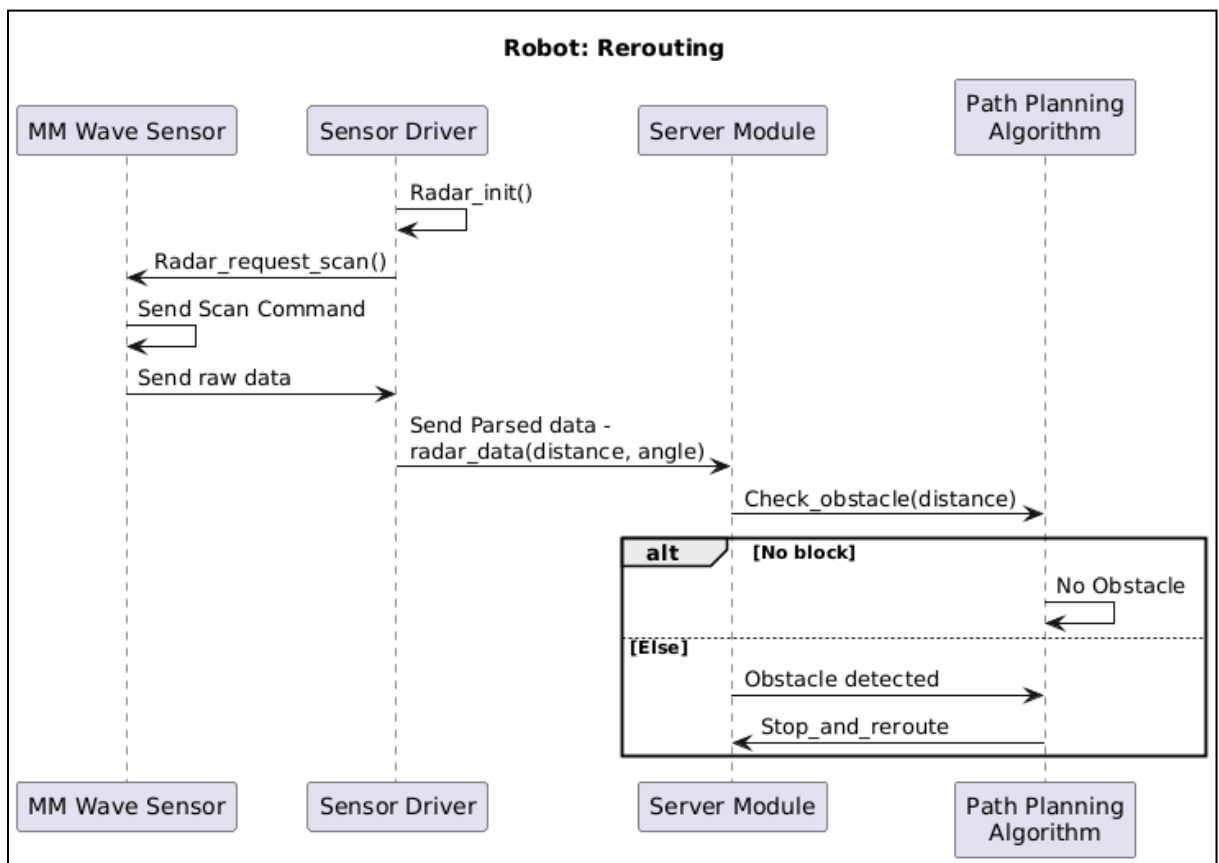


Figure 6. Rerouting

Design Element ID: D6.1 Rerouting Algorithm

Description: Executes the core logic for obstacle avoidance and path redirection. This function's algorithm is activated when an obstacle is detected by the sensor driver and MM Wave Radar, ensuring the robot can find a new path to continue its mission.

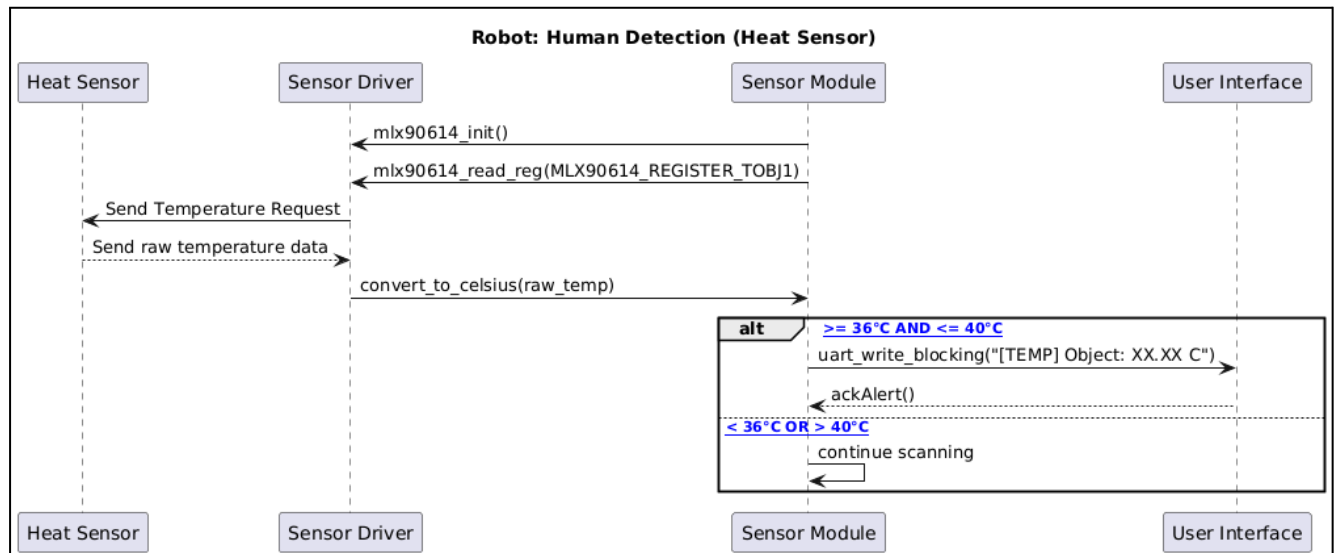


Figure 7. Human Detection (Heat Sensor)

Design Element ID: D7.1 (Heat Sensor)

Description: The Heat Sensor is a non-contact infrared temperature sensor hardware component that communicates via I2C protocol at address 0x5A. It measures both ambient and object temperatures within its field of view, responding to register read requests from the sensor driver by returning raw 16-bit temperature data in digital format.

Design Element ID: D7.2 (Sensor Driver)

Description: The Sensor Driver is a software module that abstracts I2C communication with the MLX90614 temperature sensor. It initializes the I2C interface on GPIO pins GP8 (SDA) and GP9 (SCL) at 100kHz and manages all low-level communication with the sensor. The driver reads temperature registers (0x06 for ambient temperature and 0x07 for object temperature) and converts the raw sensor data to Celsius using the formula: $(\text{raw} \times 0.02) - 273.15$. It handles I2C communication errors and timeouts to ensure reliable operation, providing a clean interface for the Control System to access temperature data without dealing with hardware-level details.

Design Element ID: D7.3 (Sensor Module)

Description: The Sensor Module is the central coordination module that orchestrates the temperature detection cycle. It calls the `read_and_send_temperatures()` function every 1 second as part of the main sensor reading interval. The Sensor evaluates the object temperature against the human body temperature range (36-40°C) to determine if a potential survivor is detected. It formats temperature data into message strings and decides when to send alerts based on temperature thresholds. The module coordinates with the LoRa module for wireless data transmission and logs all temperature readings to the console for debugging and monitoring purposes.

Design Element ID: D7.4 (User Interface)

Description: The User Interface component is a wireless communication module that uses UART0 at 9600 baud to transmit temperature data to a remote operator or monitoring station. It receives formatted messages from the Control System containing temperature readings and transmits them wirelessly to enable remote awareness of the robot's thermal sensor readings. The module operates in transparent mode (M0=0, M1=0) and sends acknowledgment signals after successful transmission to confirm that data has been delivered to the remote monitoring station.

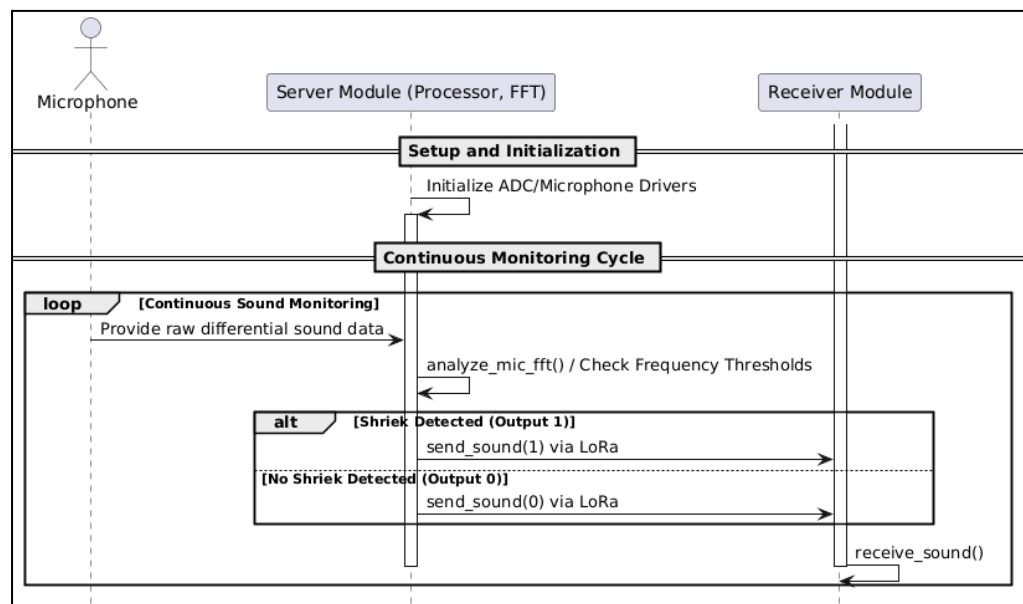


Figure 8. Microphone sensor (Detect Human Sound with Specified Frequency)

Design Element ID: D8.1- Microphone

Description: Takes in sound inputs in the form of analog values.

Design Element ID: D8.2 - Server Module

Description: This is the central module that runs the microphone driver. It initializes the Analog-to-Digital Converter (ADC) peripheral for differential microphone sampling and converts the values from analog to digital. The Fast Fourier Transform (FFT) function performs FFT on a buffer of audio samples to detect the dominant frequency and magnitude, enabling shriek classification.

Design Element ID: D8.3 - Receiver Module

Description: This is LoRa that receives values, which are then used to route the spider to the detected human.

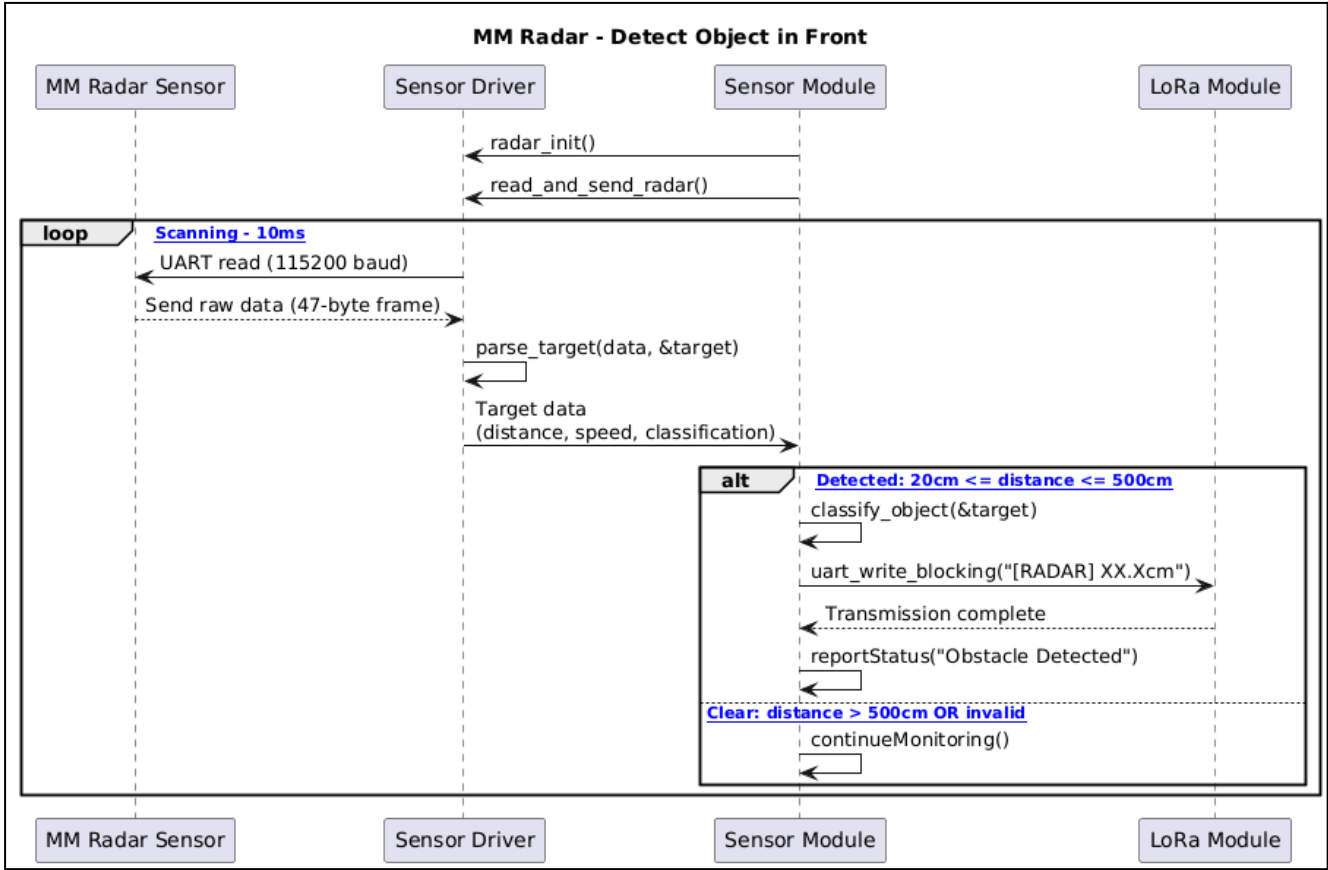


Figure 9. MM Radar (Object Detection)

Design Element ID: D9.1 (MM Radar Sensor)

Description: The MM Radar Sensor is a millimeter-wave radar hardware component used for obstacle detection and motion tracking. It communicates via UART at 115200 baud and continuously scans the environment at approximately 10Hz frame rate. The sensor can detect up to 3 simultaneous targets per frame and transmits 47-byte data frames marked with the identifier 0xFDFC. For each detected object, it provides X and Y coordinates along with speed information. The sensor operates within a detection range of 20cm to 500cm, with a blind zone below 25cm where detections are considered unreliable and filtered out.

Design Element ID: D9.2 (Sensor Driver)

Description: The Sensor Driver is a software module that manages UART1 communication with the radar sensor on GPIO pins GP4 (TX) and GP5 (RX). It continuously monitors the incoming byte stream to detect frame start markers (0xFDFC) and then parses the complete 47-byte binary frames to extract target data. The driver extracts X and Y coordinates along with speed information for up to

3 targets per frame, calculating distance using the Pythagorean theorem $\sqrt{x^2 + y^2}$ and angle using the arctangent function $\text{atan2}(y, x)$. It filters out invalid targets that fall outside the acceptable range or have invalid coordinates, maintaining proper frame synchronization while handling UART communication errors to ensure reliable data delivery to the Control System.

Design Element ID: D9.3 (Sensor Module)

Description: The Sensor Module is the central processing module that runs in a continuous 10ms polling loop to provide real-time obstacle detection. It receives parsed target data from the Sensor Driver and applies distance range filters to ensure only targets between 20cm and 500cm are considered valid. The module classifies detected objects based on their speed and motion patterns: OBJECT_HUMAN for targets with speeds between 10-500 cm/s exhibiting realistic motion, OBJECT_STATIONARY for targets with speeds below 15 cm/s and stable distance measurements, and OBJECT_NOISE for detections in the blind zone or with unrealistic speeds that are filtered out. The Sensor Module maintains a history of previous target detections to improve classification accuracy over time, makes navigation decisions based on obstacle detection results, and coordinates the transmission of alerts for valid detections to the operator.

Design Element ID: D9.4 (LoRa Module)

Description: The LoRa Module is a wireless data transmission component that uses UART0 at 9600 baud to communicate with a remote operator. It receives radar detection messages from the Control System and transmits formatted obstacle data including "[RADAR-HUMAN]" messages with distance, angle, and speed information for human detections, and "[RADAR]" messages with distance, angle, and classification data for other detected objects. The module provides real-time environmental awareness to the operator, enabling remote monitoring of the robot's obstacle detection capabilities. It confirms successful transmission back to the Control System to ensure reliable data delivery in potentially challenging communication environments.

Use Case Diagram

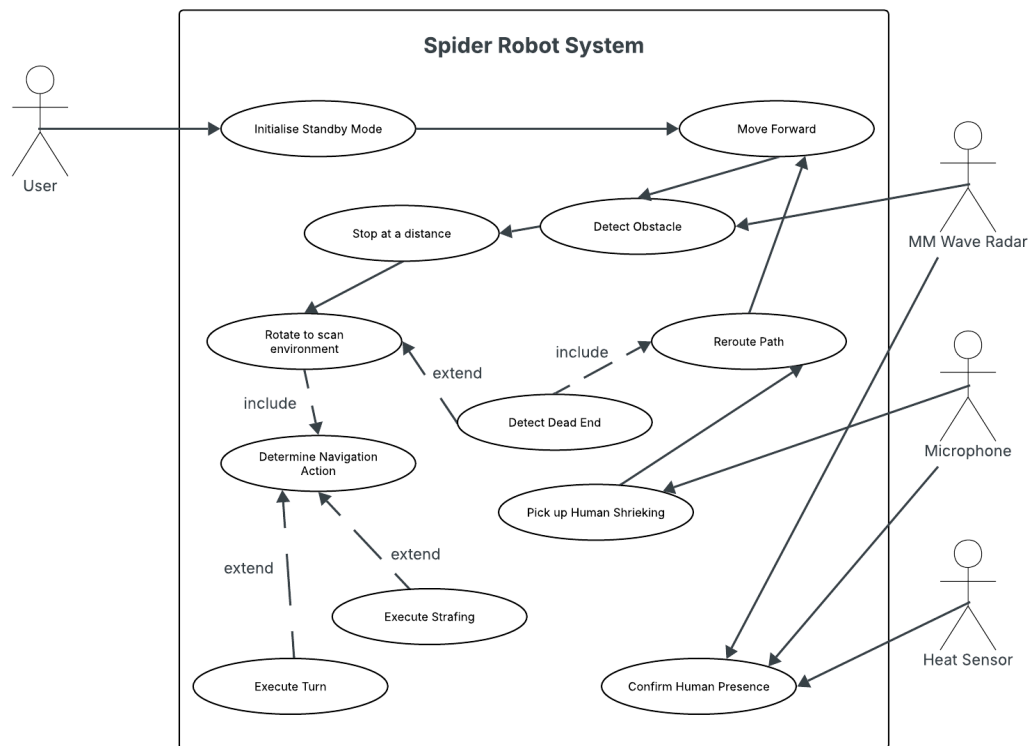


Figure 10. Use Case Diagram for Spider Robot System

The Spider Robot System Use Case Diagram illustrates how the robot interacts with its environment, sensors, and user to navigate autonomously and locate a human. The user initiates the system, after which the robot moves forward while continuously detecting obstacles using the MM Wave Radar. When an obstacle or dead end is encountered, it scans the environment, determines a new navigation path, and executes actions such as turning or strafing. The Microphone detects human shouting to guide rerouting, and the heat sensor, microphone and mm wave radar will jointly confirm human presence upon arrival.

6. Driver Interfaces

Driver Module 1: Heat Sensor Driver

1) Overview

- **Module name:** Heat Sensor (**MLX90614** Non-Contact Infrared (IR) Thermometer)
- **Purpose:** Detects human presence under rubble based on average human temperatures
- **Transport:** I²C for digital connection - SDA for sending and receiving temperature, SCL as a clock line to synchronise sending of data.
- **Blocking model:** Blocking - When function to read temperature is called, system waits until temperature is read before returning value for immediate processing
- **Timing:**
 1. Typical Latency: 0.1s
 2. Worst-case latencies: 0.84s
 3. Sampling / Update rate: will be customized to once every 1000ms
- **Units:** Temperature in °C

2) Dependencies

- Required MCU libraries: `pico/stdlib.h`, `hardware/i2c.h`, `mlx90614` (mcauser,n.d.)
 - Peripheral interfaces:
 - I2C, which enables communication with the sensor using function `i2c_init`
 - GPIO pins on SDA (Pin 8) and SCL (Pin 9)
 - UART to send processed temperature data to computer LoRa
-

3) Resources & Constraints

- Pins used, timers, DMA, IRQ priorities:
 - I²C SDA and SCL, Power: VCC (3.3V/5V) and GND
 - 1 timer for periodic sampling
 - CPU use (approx): CPU usage during temperature taking
-

4) Initialisation & Configuration (mahmoodmustafashilleh, n.d.)

- Required order of calls:
 - `Init_peripherals` → `mlx90614_init()` → ready
 - Calibration steps (if any) and time to ready:
 - Configure the SDA and SCL pins for communication. Enforce the reading interval at 100ms. Time to get ready is instantaneous.
-

5) Error Handling

- Return codes enum with meanings and common remediation (retry, re-init, backoff).
 1. OK - measurement valid
 2. EINVAL - bad parameters
 3. EBUSY - previous measurement still active, or interval was too short, to retry after minimum interval
 4. ETIMEOUT - sensor did not respond over I2C
 5. ENOINIT - driver used before initialization, to call `heat_init()` first
 6. EHW - hardware fault, to reset sensor or check wire
 - Timeouts and what happens when they fire.
 - If read fails and returns ETIMEOUT, driver clears busy state, logs last failure and allows retries after a safe interval
-

6) Test Notes

These notes are derived from FR004.

Bench Setup:

- Ambient Baseline: Log 10 samples of the ambient (room) temperature before introducing any heat source to establish a stable baseline
- Source Placement: Position the heat source human hand 5 cm away from the sensor face during measurement.
- Test Sources: Use a calibrated thermometer to confirm test source temperatures as written in FR004

Acceptance Ranges (TC-4.1 - Basic Detection):

- **Detection Differential:** The temperature reading must peak at least 5 degrees above the ambient baseline reading when the hot source is introduced.
- **Recovery Time:** The reading must return to within +1 degree of the ambient baseline within 30 seconds after the heat source is removed.
- **LoRa Consistency (TC-4.1):** The temperature value displayed on the Computer Pico must match the raw value logged by the Sensor Pico to within 0.5 degrees.

Known Limitations:

- **Ambient Drift:** Rapid changes in ambient temperature (e.g., opening a window) can affect the absolute accuracy of the reading and recovery time.
 - **Range:** The sensor's object detection range is 10cm maximum.
-

Driver Module 2: Motor Driver

Link to SRS: The system moves the robot front, back, left and right based on our input or autonomous navigation.

1) Overview

- **Module name:** 4 legged spider motor
 - **Purpose:** Main module to navigate debris. Movement of spider is done through an application, over wifi.
 - **Transport:** I2C
 - **Blocking model:** non-blocking as commands sent asynchronously
 - **Timing:**
 - Servo operating speed of 0.12 seconds/60 degrees
 - **Units:** Movement directions (front,back,left,right)
-

2) Dependencies

- Required MCU libraries
 - hardware.i2c.h
 - Peripheral interfaces (e.g., GPIO, I²C, timers, PWM):
 - I2C_PORT i2c0
 - SDA_PIN 0
 - SCL_PIN 1
 - PCA9685_ADDR 0x40
-

3) Resources & Constraints

- Memory footprint (ROM/RAM), CPU use (approx): Low CPU usage
 - Constraints:
 - Latency affected by network
-

4) Initialisation & Configuration

- Calibration steps (if any) and time to ready:
 - `stdio_init_all()` → `i2c_init()`

5) Error Handling

- Timeouts and what happens when they fire: retry sending commands. If they backfire, check I2C connections, servo connections

6) Test Notes

Test Notes are derived from FR001 to FR003.

Bench Setup

- Platform: A flat, non-slip surface to prevent drifting or unintentional rotation during tests.
- Power Supply: Stable power source to ensure consistent servo performance, a powerbank
- Measurement: Distance markers on the floor

Acceptance Ranges

Criterion	Requirement	Source FR
Default Posture	Robot must immediately settle into the <code>SPIDER_STANDBY</code> pose upon power-on. Posture must remain stable (no shaking or drifting).	FR-001
Motion	Robot must successfully move in the intended direction (Forward, Backward, Left, Right) when commanded.	FR-002
Rotation	Robot must successfully rotate in the intended direction (Clockwise/Right, Counter-Clockwise/Left) when commanded.	FR-003
Default Pose	After completing any movement or rotation command, the robot must default back to the <code>SPIDER_STANDBY</code> pose without further command.	FR-002, FR-003

Fault Injection and Stress Testing

- Simulate a servo error (e.g., unplugging one servo channel) upon startup.
 - Expected Result: System should detect an error state or stop all movement; it should not attempt movement with missing limbs.

Known Limitations

- **Surface Constraint:** Speed and stability is dependent on surface friction.
- **Weight Constraints:** Added weight can affect speed and gait flexibility

Driver Module 3: Radar Driver

1) Overview

- **Module name:** 1 Millimeter radar LD 2450
 - **Purpose:** Detect obstacles around the robot and provide real-time data to support navigation towards humans in rubble
 - **Transport:** UART (default: 256,000 baud, 1 stop bit, no parity) (HiLink, 2023)
 - **Blocking model:** Non-blocking
 - **Timing:**
 1. Typical Latency: Approximately 100 ms per detection cycle
 2. Worst-case Latency: Up to 200 ms under optimal conditions
 3. Sampling/Update Rates: 10 Hz (data refresh rate) (HiLink,2023)
 - **Units:** Distance: Millimeters (mm), Angle: Degrees (°)
-

2) Dependencies

- Required MCU libraries: hardware/uart.h function for communication between PICO and radar. (Waveshare, n.d.)
 - Peripheral interfaces: UART TX/RX pins in radar
 - OS services if used (e.g., FreeRTOS delays, semaphores).
-

3) Resources & Constraints

- Pins used:
 - RADAR_UART_ID uart1
 - RADAR_TX_PIN 4
 - RADAR_RX_PIN 5
 - RADAR_BAUD_RATE 115200
- Memory footprint (ROM/RAM), CPU use (approx): CPU usage is low, with most usage during data gathering and processing
- Constraints: (HiLink, 2023)

Target application	Detection and tracking of up to three targets
Detection distance	6m
Detection angle	±60 °
Data refresh rate	10Hz
Sweep bandwidth	250MHz Compliant with FCC, CE, Commission-free certification standards

4) Initialisation & Configuration

- Required order of calls: init_peripherals→ radar_init() → ready
- Calibration steps (if any) and time to ready.
 - Configure UART pins for communication
 - Time to ready is immediate

5) Error Handling

- Return codes enum with meanings and common remediation (retry, re-init, backoff)
 - OK: Measurement valid
 - EINVAL: Invalid configuration or parameters
 - EBUSY: Previous measurement still active or interval not elapsed
 - ETIMEOUT: No data received within timeout period
 - EHW: Hardware error (e.g., UART communication failure)
 - ENOINIT: API used before initialization
- Timeouts and what happens when they fire.
 - When ETIMEOUT is received, the driver to clear 'busy' state and record the last data before failure.
 - Remediation to move back 50-100mm, ensure wiring and UART are working

6) Test Notes

The test notes are derived from functional requirements (FR-006).

Bench Setup

- Distance Targets: Use static, solid obstacles (e.g., 30cm by 30cm cardboard) at the boundary test points.
- Movement Target: Use a human walking target for classification tests.
- Logging: Log detection samples (distance, speed, stability, classification) for all tests.
- Timing: Log time from detection to successful UART/LoRa message transmission for latency checks.

Acceptance Ranges

Criteria	Range	Requirement	Source
Distance Error	Less than or equal to 10cm	Absolute distance measurement accuracy for targets 0.2m to 3m.	TC-6.1
Classification (Human)	Detected Speed more than 50cm/s	Must correctly classify moving targets (e.g., human walking at 50cm/s)	TC-6.3
Classification (Stationary)	Detected Speed less than 15cm/s	Must classify object as static after history window is established.	TC-6.4

Distance Stability (Jitter)	Stationary objects for over 5 seconds	Distance measurement consistency for stationary objects over 5 seconds.	TC-6.4
End-to-End Latency	Less than 200m/s	Time from target appearance/movement to successful LoRa message transmission.	TC-6.1

Boundary and Fault Injection

- **Blind Zone Check:**
 - Test target placement at 15cm.
 - Expected Result: System must **reject detection** or classify as object_noise (Blind Zone is approximately 25cm). (Referenced in TC-6.2)
- **Maximum Range Check:**
 - Test target placement at 300cm and 500cm.
 - Expected Result: System must detect at 300cm. The system must **detect at 500cm** but **fail to detect** targets placed beyond 500cm. (Referenced in TC-6.2)
- **LoRa/UART Failure:** Inject UART errors (e.g., disconnect LORA_UART_ID) → Expect system to gracefully handle write failure without crashing the main loop.

Known Limitations

- **Radar Reflection:** Poor or noisy returns on materials with low reflectivity (e.g., fabric, soft materials, highly angled surfaces).
- **Distance Resolution:** The sensor's range resolution dictates how closely two objects can be spaced along the radial direction before they are reported as a single target.

Driver Module 4: Microphone Driver

1) Overview

- **Module name:** Microphone - Grove analog Microphone
- **Purpose:** Capture human sounds for easier location
- **Transport:** ADC
- **Blocking model:** non-blocking, as noise is sampled in periodic intervals
- **Timing:** 1000ms
- **Units:** decibels

2) Dependencies

- Required MCU libraries: kissfft for shriek detection
 - Peripheral interfaces: GPIO, ADC
-

3) Resources & Constraints

Pins used, timers, DMA, IRQ priorities.

- ADC_PIN_P 26 to MIC_P
- ADC_PIN_N 27 to MIC_N
- ADC_CHANNEL_P 0
- ADC_CHANNEL_N 1
- SAMPLE_RATE_HZ 8000
- SHRIEK_FREQ_THRESHOLD 1500
- SHRIEK_MAG_THRESHOLD 300

Memory footprint (ROM/RAM), CPU use (approx): small memory footprint, with CPU usage when sensing sound

4) Initialisation & Configuration

- Required order of calls: `init_peripherals` → `adc_mic_init()`
 - Time to ready: immediate after initialisation
-

5) Error Handling

- Return codes enum with meanings and common remediation:
 - `ADC_OK` → Sample valid
 - `ADC_TIMEOUT` → No sample acquired in expected time
 - `ADC_OVERFLOW` → DMA buffer overflowed
 - `ADC_NOISE` → Signal below threshold (likely background noise)
 - Timeouts and what happens when they fire: timeout is at 2x the expected period, and if timeout occurs, pico to retry ADC
-

6) Test Notes

This information is derived from FR-005

Bench Setup

- **Acoustic Source:** Use a speaker connected to a phone playing sine tones from a reliable YouTube source.
- **Reference Volume:** Establish a fixed, reproducible maximum volume setting of speaker volume at 75% for all high-level tests.
- **Standard Test Tone:** 1500hz within human range
- **Reference Noise Level:** Play ambient noise white noise at a controlled low volume of 20%.
- **Distances:** Primary test distances are 1m-5m
- **Logging:** Log the detected primary frequency, magnitude output from `analyze_mic_fft()`, and time from sound onset to successful LoRa alert.

Acceptance Ranges

Criterion	Range / Threshold	Requirement	Source TC
Detection Frequency Range	300Hz - 3000Hz	The primary frequency band to be classified as a valid "human range" sound.	TC-5.1, TC-5.2
Magnitude Threshold	Magnitude > 300 (at fixed reference volume)	The FFT output magnitude must cross a required software threshold when the standard tone is played at the Reference Volume .	TC-5.4
End-to-End Latency	Less than 2s	Time from tone playback start until successful LoRa alert message transmission.	TC-5.1
Noise Rejection	Zero False Positives	No alerts must be sent during periods when ambient noise is played at the Reference Noise Level .	TC-5.4

Boundary and Fault Injection

- **Frequency Boundary Check (Low):**
 - Test tone placement at 300Hz (Pass) and 250Hz (Fail).
 - Expected Result: System must detect and alert for 300Hz. The system must ignore 250Hz. (Referenced in TC-5.2)
- **Frequency Boundary Check (High):**
 - Test tone placement at 3000Hz (Pass) and 3100Hz (Fail).
 - Expected Result: System must **detect and alert** for 3000Hz. The system must **ignore** 3100Hz. (Referenced in TC-5.2)

Known Limitations

- **Volume Mismatch:** Results are dependent on the specific speaker and its volume settings, making them non-transferable to other setups.
- **FFT Resolution:** The resolution of the FFT limits the ability to precisely differentiate between closely spaced frequencies.
- **Dynamic Range:** The system may fail to accurately detect signals if the sound level is significantly below the noise floor or above the microphone's saturation limit.

7. Detailed Design

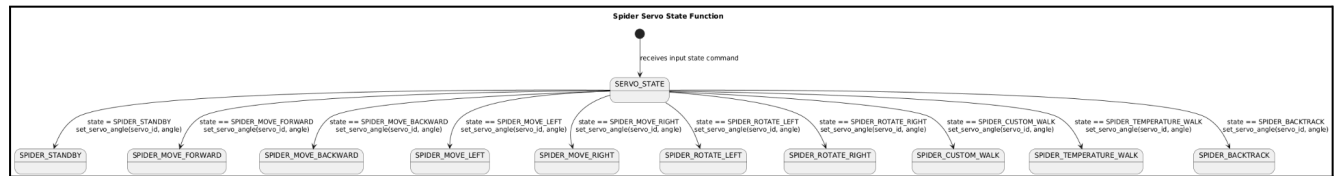


Figure 11. Servo Leg State Transition Chart (State Machine Chart)

Design Element ID: D2.1.1

Function: servo_leg_state(int state)

Description:

- Rotates the respective servos to their respective angles based on what state is given in the parameter of the function
- States are defined in the code file such as SPIDER_STANDBY, SPIDER_MOVE_FORWARD
- After completing the state animation, the function will sleep as long as the wait_time given in the parameter of the function if buffer time is needed before transiting to next state
- The function contains a set_servo_angle (servo_id, angle) function that sets a specific servo to a specified angle.
- Each state has a defined affected servos and its sequence of angles to rotate to that that the spider legs will perform the state animation

Design Element ID: D2.1.2

Function: set_servo_angle (servo_id, angle)

Description:

- Sets a specific servo to a specified angle
- Inside each state contains multiple set_servo_angles that sets a different servo to rotate and hold to a specific angle to form the desired pose

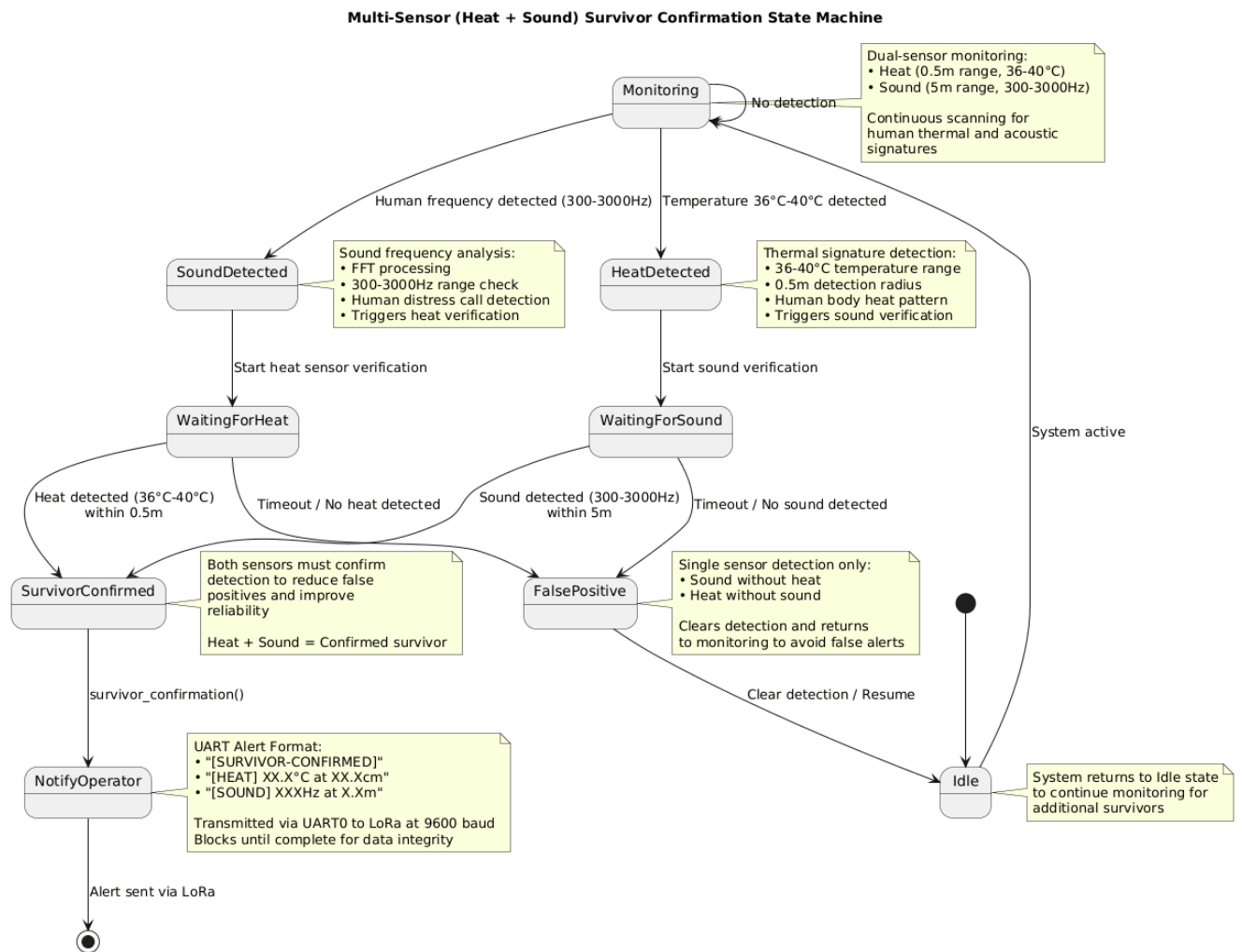


Figure 12: Multi-Sensor (Heat + Sound) Survivor Confirmation State Machine

Design Element ID: D7.2.1

Function: read_temperature()

Description:

- Reads ambient and object temperature from MLX90614 sensor:
 - Ambient temperature from register 0x06
 - Object temperature from register 0x07
- Converts both readings to Celsius using convert_to_celsius().
- Checks if object temperature falls within human thermal range (36°C–40°C).
- Returns 1 if heat signature is detected within this range, otherwise 0.
- Used in HeatDetected and WaitingForHeat states to verify presence of a warm body.
- Verification must succeed within a 5-second timeout; failure triggers transition to FalsePositive.
- Executed once per second during verification.
- Does NOT transmit LoRa; final alert is generated only in the NotifyOperator state.

Design Element ID: D8.2.1

Function: analyze_mic_fft() - Confirmation Context

Description:

- Performs FFT-based frequency extraction from microphone samples.

- Detects human distress frequency in the range of 300–3000 Hz.
- Returns 1 if dominant frequency falls within range and magnitude threshold is exceeded.
- Used in SoundDetected and WaitingForSound states to verify sound presence.
- Verification must complete successfully within a 5-second timeout.
- If no valid sound is detected before timeout, system transitions to FalsePositive.
- Does not send alerts; contributes only one confirmation signal.
- Executed only during sound verification periods, not continuously.

Design Element ID:D7.2.5

Function: survivor_confirmation()

Description:

- Called when both heat and sound verification succeed.
- Encapsulates final decision logic confirming survivor presence.
- Checks:
 - heat_detected == 1
 - sound_detected == 1
- Returns TRUE only when both sensors confirm human presence.
- If TRUE → transitions system to NotifyOperator state.
- If FALSE → transitions system to FalsePositive state.
- Reduces false positives and ensures high-confidence detection.
- Triggers the transmission of survivor alert via UART LoRa module.
- Executed once in the SurvivorConfirmed state.

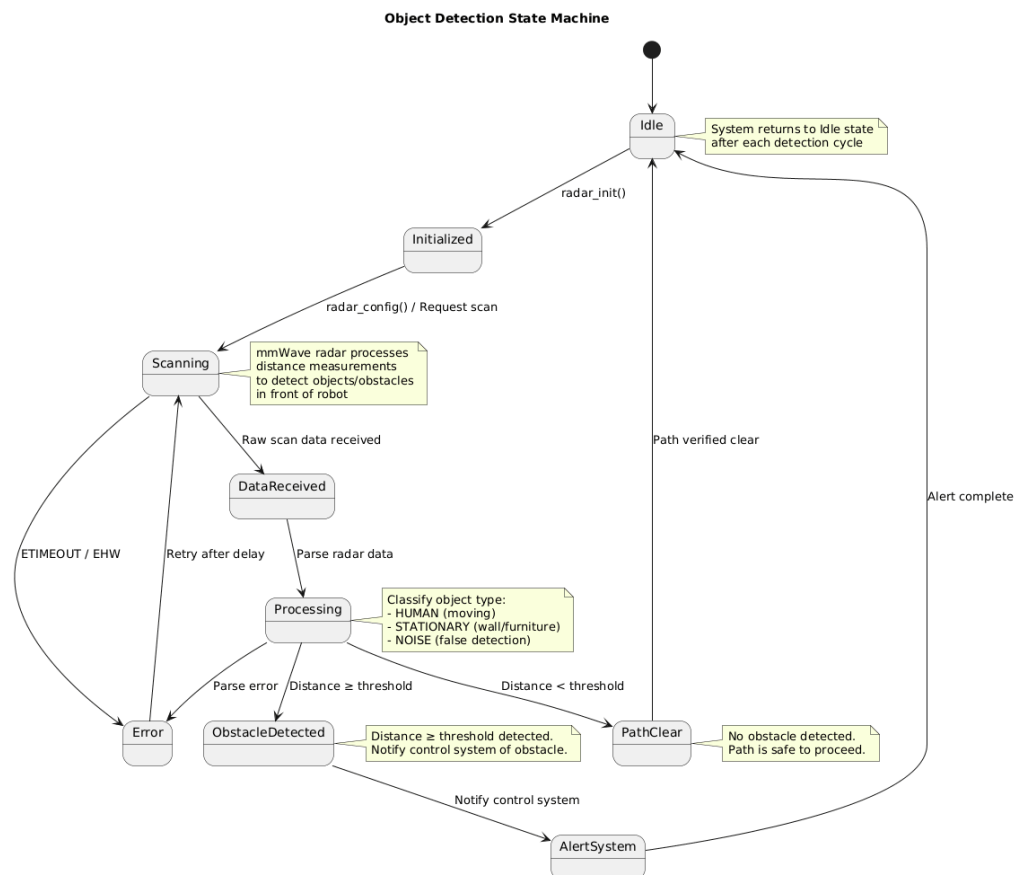


Figure 13: Object Detection State Machine

Design Element ID: D9.2.1

Function: radar_init()

Description:

- Initializes mmWave radar communication prior to Scanning state.
- Configures UART1 at 115200 baud for radar data reception.
- Sets GPIO 4 as UART1 TX and GPIO 5 as UART1 RX with no hardware flow control.
- Applies 100ms stabilization delay to allow radar module to boot.
- Clears internal radar buffers and resets frame counters.
- Verifies UART readiness and logs initialization status.
- Upon successful initialization, system transitions from Idle → Initialized state.
- If initialization fails, system stays in Idle and logs an error.

Design Element ID: D9.2.2

Function: read_and_send_radar()

Description:

- Runs during the Scanning state to acquire a single radar data frame.
- Reads bytes from UART1 until detecting the frame start marker 0xFD FC.
- Accumulates full radar frame (47 bytes) into receive buffer.
- Validates frame length and checksum if present.
- If frame is incomplete or UART times out, Scanning transitions to Error state.
- When a valid frame is fully received, system transitions Scanning → DataReceived.
- Does not classify objects or send alerts; only handles frame acquisition.
- Reads occur once per detection cycle, not continuously.

Design Element ID: D9.2.3

Function: parse_target(uint8_t *data, Target *t)

Description:

- Executes in the Processing state to convert raw radar bytes into meaningful target data.
- Extracts signed 16-bit X coordinate from bytes 0–1.
- Extracts signed 16-bit Y coordinate from bytes 2–3.
- Extracts signed 16-bit relative speed from bytes 4–5.
- Computes Euclidean distance in millimeters using $\sqrt{x^2 + y^2}$.
- Computes angular direction using $\text{atan2}(y, x)$ in degrees.
- Populates Target structure with distance, angle, and speed.
- Called for each potential target (up to 3 targets per frame).
- If parsing fails or values are invalid, Processing transitions to Error state.

Design Element ID: D9.2.4

Function: classify_object(Target *target, Target *prev_targets, int history_count)

Description:

- Runs in the Processing state after target parsing.
- Converts distance to centimeters for threshold comparison.
- Applies blind-zone rejection: distance < 25 cm → OBJECT_NOISE.
- Computes instantaneous speed from current and previous frame distances.
- Computes average velocity and distance stability using last N frames.
- Classifies object based on motion characteristics:
 - OBJECT_STATIONARY: speed < 15 cm/s and stable distance
 - OBJECT_HUMAN: speed 10–500 cm/s with consistent motion

- OBJECT_NOISE: speed > 500 cm/s or unstable measurements
- Returns object_type_t enumeration.
- If classified object distance < threshold, system transitions → ObstacleDetected.
- If no valid object or distance ≥ threshold, transitions → PathClear.

Design Element ID: D9.2.5

Function: uart_write_blocking() - Radar Alert

Description:

- Executed during the AlertSystem state to notify higher-level systems of detected obstacles.
- Formats classified radar results into human-readable message strings:
 - HUMAN: "[RADAR-HUMAN] XX.Xcm XX.Xdeg XX.Xcm/s"
 - STATIONARY: "[RADAR] XX.Xcm XX.Xdeg STATIONARY"
 - NOISE: "[RADAR] NOISE"
- Transmits message via UART0 to the LoRa module at 9600 baud.
- Blocks execution until message fully transmitted to ensure reliability.
- After successful alert transmission, system transitions AlertSystem → Idle.
- If transmission fails, logs communication error but still returns to Idle.

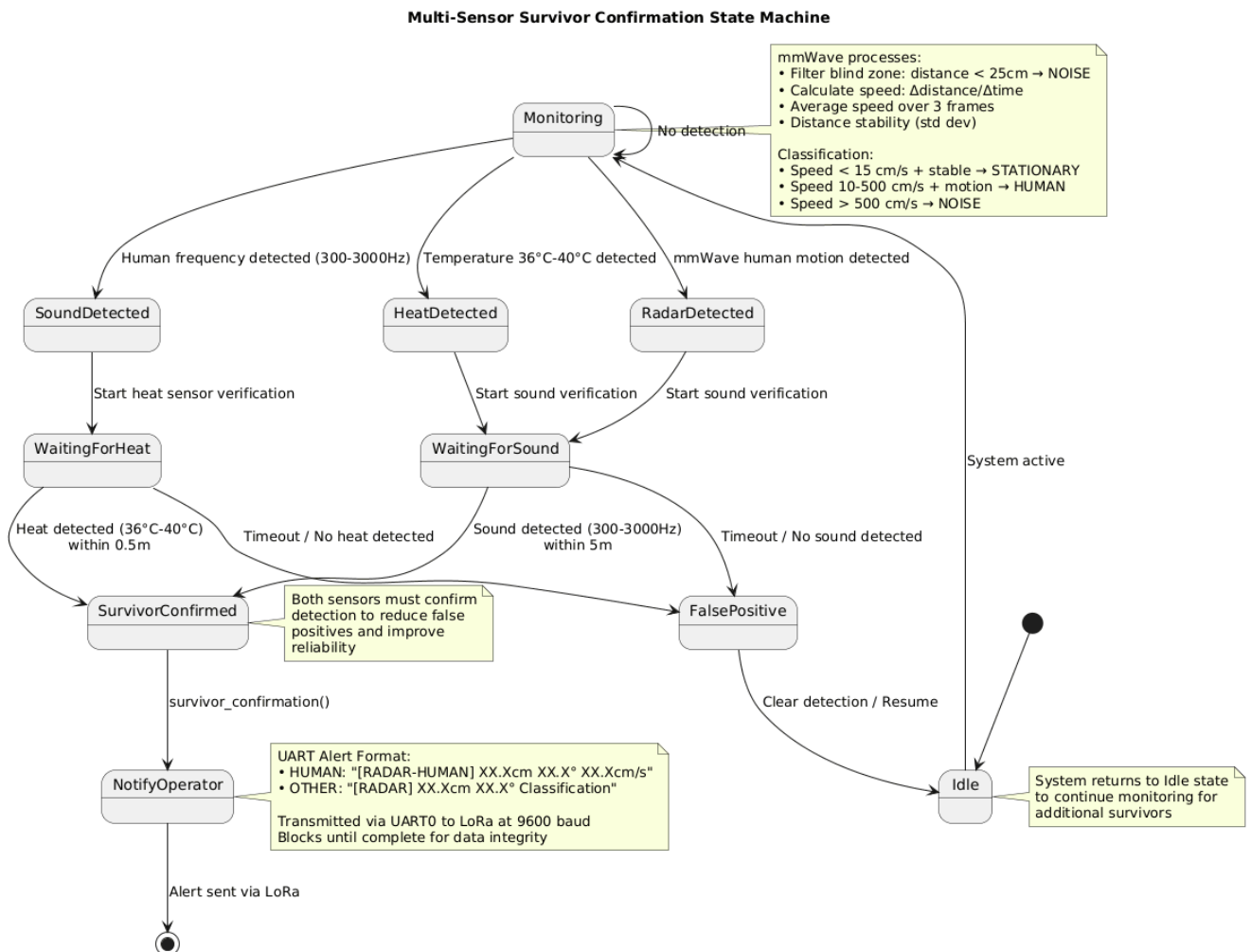


Figure 14: Path Routing to Survivors State Machine

Design Element ID: D3.6.1

Function: read_temperature()

Description:

- Reads both ambient and object temperature from MLX90614:
 - Ambient temperature from register 0x06
 - Object temperature from register 0x07
- Converts raw sensor values to Celsius using convert_to_celsius().
- Validates object temperature range for human presence (36°C–40°C).
- Returns detection flag: 1 = heat detected, 0 = no heat detected.
- Used in HeatDetected and WaitingForHeat states to verify presence of a nearby warm body.
- Does NOT transmit LoRa; final alert is sent only from NotifyOperator state.
- Supports timeout logic: if no valid heat reading is detected within 5 seconds, system transitions to FalsePositive.
- Sampling rate: executed every 1 second during monitoring cycle.

Design Element ID: D3.6.2

Function: analyze_mic_fft() - Confirmation Context

Description:

- Collects audio samples and performs FFT-based frequency analysis.
- Detects human distress frequency in the range 300–3000 Hz.
- Returns 1 if dominant frequency is within range and magnitude threshold is exceeded.
- Used in SoundDetected and WaitingForSound states to verify sound presence.
- Must provide a positive detection within a 5-second verification window.
- If no valid sound is detected within timeout, system transitions to FalsePositive.
- Does not send alerts directly; contributes one of the confirmation signals.
- Executed only when sound verification is required, not continuously.

Design Element ID: D3.6.3

Function: classify_object() - Human Classification

Description:

- Processes mmWave motion data to determine if a moving human is present.
- Evaluates:
 - Instantaneous speed derived from distance change
 - Average speed over last 3 frames
 - Distance stability (standard deviation)
- Applies blind-zone filtering (distance < 25 cm → NOISE).
- Classifies detections into:
 - HUMAN (10–500 cm/s + motion signatures)
 - STATIONARY
 - NOISE
- Returns OBJECT_HUMAN when radar indicates human motion.
- Used in RadarDetected and WaitingForSound states to provide radar-based confirmation.
- Acts as an alternative verification path when sound detection fails.
- Does not output alerts; contributes to multi-sensor confirmation logic.
- Executed only when radar detects a moving object.

Design Element ID:D3.6.4

Function: survivor_confirmation()

Description:

- Evaluates detection results from the three available sensors:
 - Heat detection flag (36–40°C within 0.5 m)
 - Sound detection flag (300–3000 Hz distress frequency)
 - Radar human-motion classification (OBJECT_HUMAN)
- Implements multi-sensor fusion with majority agreement.
- Valid combinations considered positive:
 - Heat + Sound
 - Heat + Radar
 - Sound + Radar
- Returns TRUE if ≥ 2 sensors detect a human, otherwise FALSE.
- Executed in the SurvivorConfirmed state after both verification sequences complete.
- If TRUE:
 - System enters NotifyOperator state to transmit survivor alert.
- If FALSE:
 - System enters FalsePositive state and returns to Idle.
- Reduces false positives, improving reliability of survivor detection.
- Ensures system only alerts operator when confidence is high.

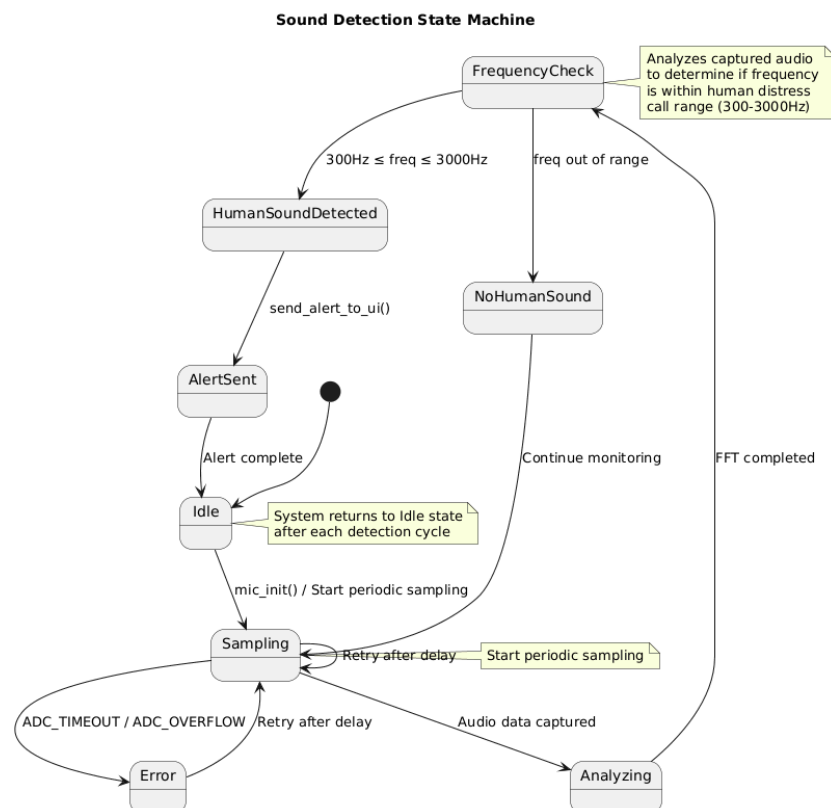


Figure 15: Sound Detection State Machine

Function: `adc_mic_init()`

Description:

- Initializes the ADC subsystem required for microphone audio sampling.
- Configures GPIO pin 26 as ADC channel 0 (MIC_P) and GPIO pin 27 as ADC channel 1 (MIC_N).
- Enables differential microphone mode by sampling MIC_P and MIC_N consecutively to reduce noise.
- Sets ADC resolution to 12-bit mode.
- Configures ADC sample rate timing to support 8 kHz microphone sampling.
- Called when transitioning from Idle → Sampling at the start of each detection cycle.
- Must complete successfully before entering Sampling state.
- If initialization fails, the system transitions to Error state.

Design Element ID: D8.2.3

Function: `read_microphone()`

Description:

- Reads a single differential microphone sample during the Sampling state.
- Performs ADC conversion on channel MIC_P (ADC0) followed by MIC_N (ADC1).
- Computes differential audio value: $\text{sample} = (\text{MIC_P} - \text{MIC_N})$.
- Offsets sample by +2048 to center the waveform in the unsigned 12-bit range.
- Called repeatedly (128 times) to fill one audio frame for FFT analysis.
- Sampling interval aligns with 8 kHz sample rate.
- If ADC does not respond or returns overflow/timeout, Sampling transitions to Error state.
- Returns the differential audio sample as a 16-bit signed integer.

Design Element ID: D8.2.4

Function: `analyze_mic_fft()`

Description:

- Executed during the AlertSent state after a HumanSoundDetected event.
- Formats the FFT detection result as: "[MIC] Shriek: X" where X = 1 for detected distress or 0 otherwise.
- Transmits the message to the LoRa module via UART0 at 9600 baud.
- Transmission is blocked to guarantee that the alert is delivered before transitioning to Idle state.
- May also print the message to the local console for debugging.
- Successful completion triggers AlertSent → Idle transition.

Design Element ID: D8.2.5

Function: `uart_write_blocking()` - Microphone Alert

Description:

- Formats detection result into string: "[MIC] Shriek: X" where X is 0 or 1
- Transmits message via UART0 to LoRa module at 9600 baud
- Blocks execution until complete message transmitted (ensures delivery)
- Executed in AlertSent state after frequency analysis completes
- Also logs result to console via `printf()` for debugging

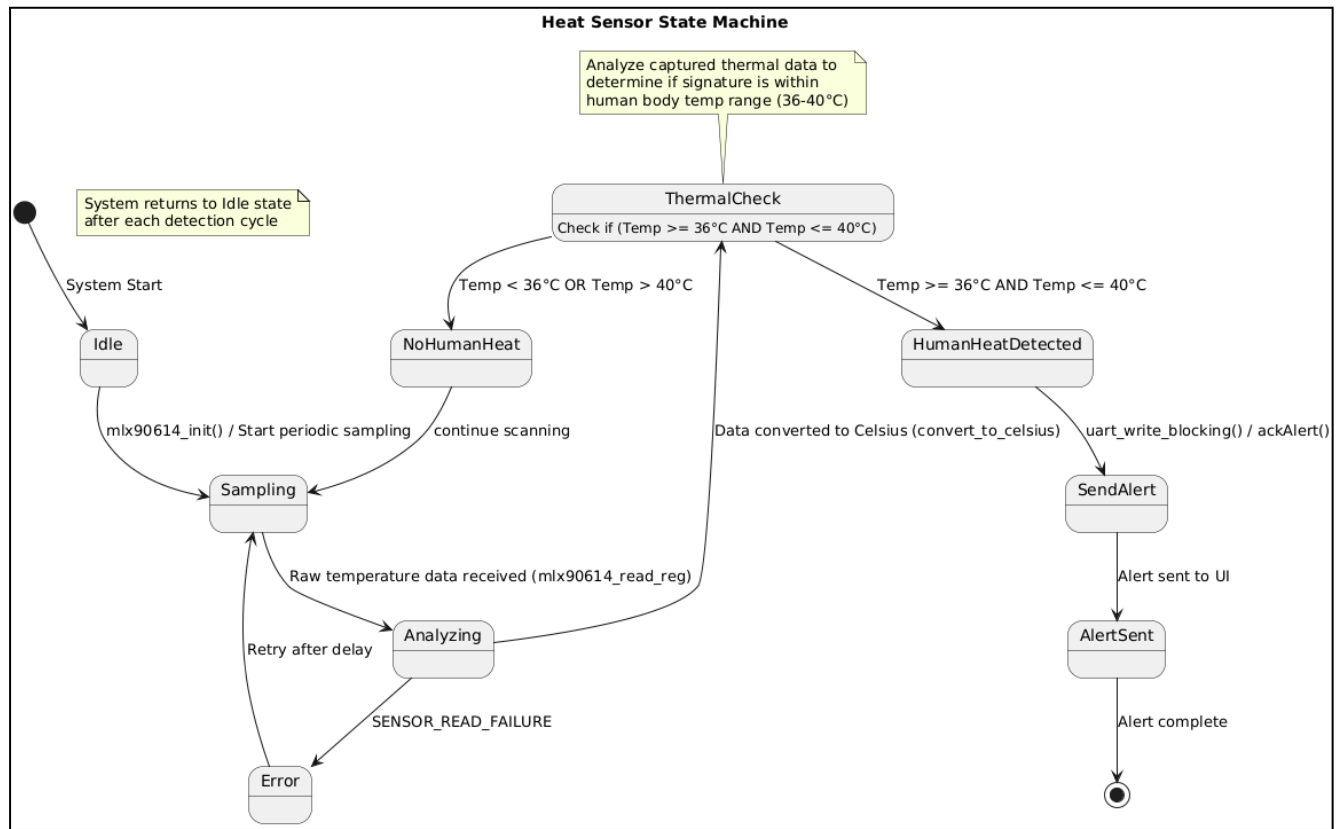


Figure 16. Heat Sensor State Chart

Design Element ID: D7.2.2

Function: `mlx90614_init()`

Description:

- Initialize the heat sensor
- Sets up the I²C connection between the Sensor Pico and the heat sensor

Design Element ID: D7.2.3

Function: `mlx90614_read_reg()`

Description:

- Reads the raw temperature data being recorded by the heat sensor
- Reads both an object raw temperature data as well as the ambient raw temperature data

Design Element ID: D7.2.4

Function: `convert_to_celsius(raw_temp)`

Description:

- Converts the raw temperature data to degree celsius

Design Element ID: D7.4.1

Function: `uart_write_blocking()` - Human Heat Alert

Description:

- Send alerts to the Computer Pico to flag out that human heat has been detected by the heat sensor

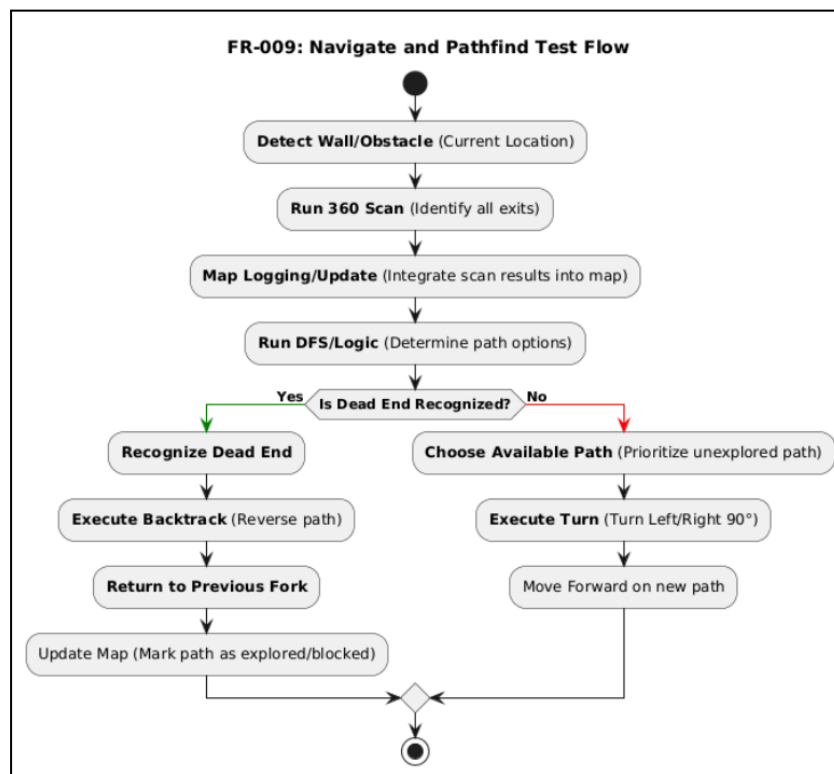


Figure 17. Flowchart for Navigation and Path Finding

Design Element ID: D3.3.1

Function: `Sensors_parse_radar_string()`, `Control_execute_action()`, `Fork_scan_paths()`

Description:

- Initiates the 360° environment scan (`is_scanning = true`) when a WALL is detected.
- The process requires a `COMMAND_CRIT_STANDBY` to read the initial sensor data, followed by three consecutive `COMMAND_MOVE_SCAN` actions.
- Each scan command simulates a 90° rotation (via `SPIDER_ROTATE_RIGHT`) and increments `scan_steps`.
- During the scan, `Fork_scan_paths` reads the sensor status at each 90° interval, records new unexplored open paths (`PATH_UNEXPLORED`) detected by `EMPTY_SPACE_DETECTED` (or `CLEAR_PATH`), and ignores the path of entry and the wall that triggered the fork creation.

Design Element ID: D3.1.1

Function: `DFS_decision_maker()`, `stack_push()`, `next_available_fork_id`

Description:

- Implements Fork/Junction Creation. When a WALL is detected while walking, the current state (`parent_fork_id`, `path_index`, and `entry_pose`) is pushed onto the DFS stack.
- The robot then assumes it is at a new fork (`current_fork_id` is incremented).
- The path directly in front of the robot (where the wall was found) is explicitly initialized and marked as `PATH_DEAD_END` (Path 0) before the 360° scan begins.

Design Element ID: D3.4.1

Function: `DFS_decision_maker()`, `Fork_find_unexplored_path()`, `Sensors_get_angle_to_turn()`, `Control_execute_action()`

Description:

- Handles Path Selection and Alignment. After the scan completes, `Fork_find_unexplored_path` identifies the next available `PATH_UNEXPLORED`.
- This path's status is set to `PATH_EXPLORING`. `Sensors_get_angle_to_turn` calculates the shortest angle required to align the robot's current heading with the path's `target_heading_deg`.

- The robot then executes COMMAND_MOVE_TURN_LEFT (for positive angle) or COMMAND_MOVE_TURN_RIGHT (for negative angle) until aligned.

Design Element ID: D3.4.2

Function: DFS_decision_maker(), stack_pop(), Control_execute_action()

Description:

- Manages Dead End Backtracking. If a fork has been fully scanned and Fork_find_unexplored_path returns -1 (no paths left), the fork is a dead end.
- The robot pops the parent state from the DFS stack, marking the path it entered on the parent fork as PATH_DEAD_END.
- The robot then executes COMMAND_MOVE_BACKTRACK, which simulates a 180° rotation followed by a walk (SPIDER_BACKTRACK) to return the robot to the entry position of the parent fork.

Design Element ID: D3.5.1

Function: DFS_decision_maker() (switch cases), Control_execute_action()

Description:

Implements Priority Obstacle/Goal Handling. Specific sensor detections override the standard DFS walk/scan logic:

- STRAFE_OBJECT: Triggers COMMAND_MOVE_STRAFE (simulated sideways movement).
- POSSIBLE_HUMAN: Triggers COMMAND_TEMPERATURE_WALK (custom forward gait).
- HUMAN_CONFIRMED or GOAL_OBJECT: Triggers COMMAND_MOVE_HALT or COMMAND_STOP_PROGRAM, marking the current path as PATH_SUCCESS to indicate mission completion.

Unified Sensor Failure Flowchart

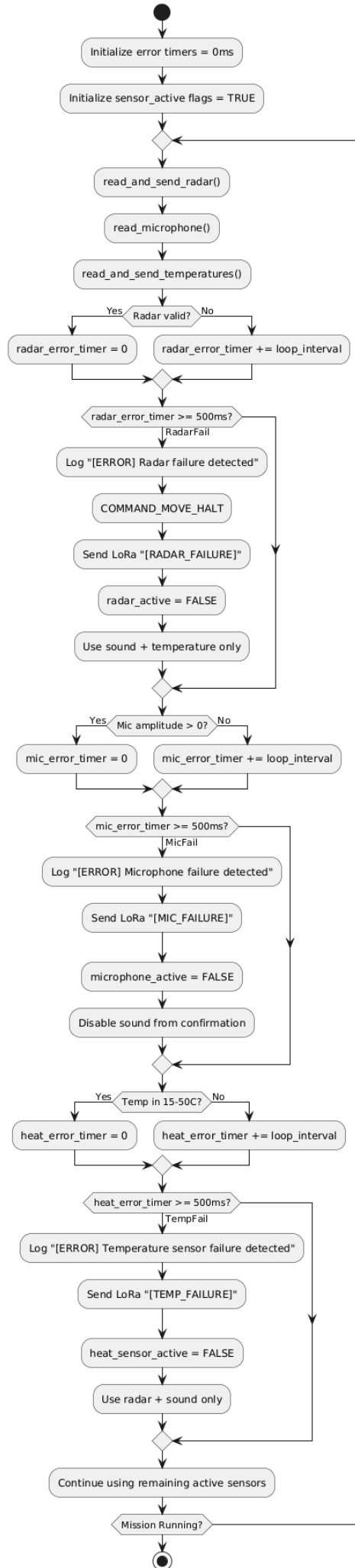


Fig 19. Flowchart to Handle Sensor Failure

Design Element ID: D9.3.1

Function: read_and_send_radar(), sensor_fail_handler(RADAR_ID)

Description:

- Log “[ERROR] Radar failure detected”
- Execute COMMAND_MOVE_HALT to momentarily stop movement
- Transition to RadarError transient state
- Send LoRa alert “[RADAR_FAILURE]”
- Disable radar subsystem (radar_active = FALSE)
- Resume navigation using remaining sensors: temperature and microphone
- Radar data is excluded from human detection and obstacle mapping
- Navigation continues without speed reduction or SAFE_MODE
- Radar failure counter resets only when valid radar is restored

Design Element ID: D8.3.2

Function: read_microphone(), sensor_fail_handler(MICROPHONE_ID)

Description:

- Log “[ERROR] Microphone failure detected”
- Send LoRa alert “[MIC_FAILURE]”
- Disable microphone subsystem (microphone_active = FALSE)
- Remove microphone from survivor confirmation logic
- Continue mission using radar and temperature sensors only
- No movement halt is performed
- System does not enter SAFE_MODE
- Navigation patterns and gait remain unchanged

Design Element ID: D7.3.2

Function: read_and_send_temperatures(), sensor_fail_handler(HEAT_SENSOR_ID)

Description:

- Log “[ERROR] Temperature sensor failure detected”
- Send LoRa alert “[TEMP_FAILURE]”
- Disable temperature subsystem (heat_sensor_active = FALSE)
- Continue mission without temperature-based validation
- Survivor confirmation relies on radar human signatures and sound-only detection
- No movement halt is performed
- System does not enter SAFE_MODE
- Navigation is unaffected by heat sensor failure

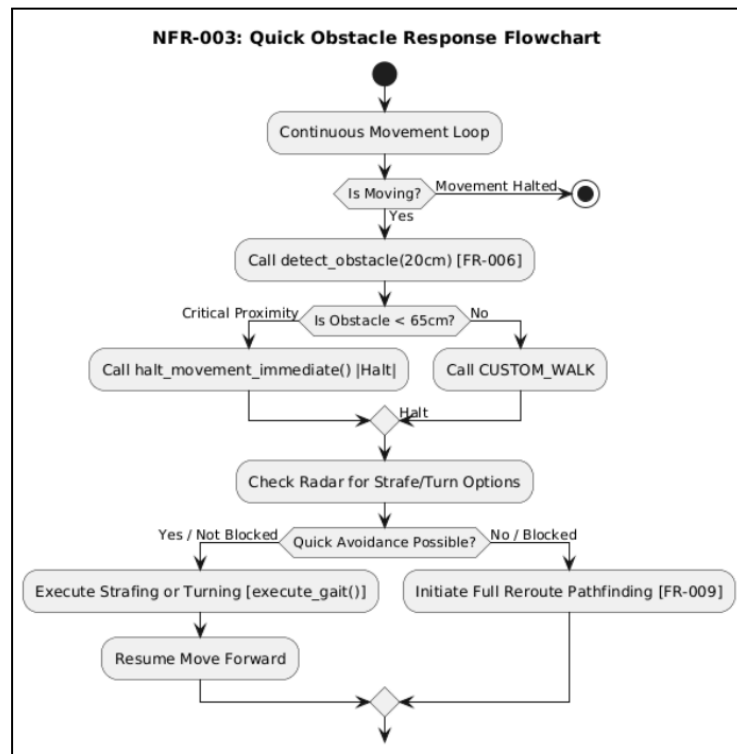


Fig 20. Flowchart for Obstacle Response

Design Element ID: D2.8.1

Function: DFS_decision_maker(), Control_execute_action(), Fork_scan_paths()

Description:

- If WALL is detected and obstacle ≥ 0.66 , the system returns COMMAND_CUSTOM_WALK.
- If STRAFE_OBJECT is detected: The system returns COMMAND_MOVE_STRAFE.
- If WALL is detected and obstacle < 0.66 m: Robot halts implicitly by initiating the is_scanning state, leading to COMMAND_CRIT_STANDBY (halt prior to scan).

Design Element ID: D3.4.3

Function: stack_pop(), Fork_update_path_status(), Control_execute_action()

Description:

- Covers the deliberative, complex path recalculation flow specific to the DFS algorithm.
- The system calls stack_pop() to retrieve the parent fork's state.
- The function Fork_update_path_status() is called to mark the current path (which led to the now-failed fork) as PATH_DEAD_END in the parent fork's map data.
- The global state (current_fork_id, current_path_number, and current_pose) is reset to the parent fork's entry state.
- The system returns COMMAND_MOVE_BACKTRACK, which executes a 180° turn and moves forward to begin retreating from the dead end.

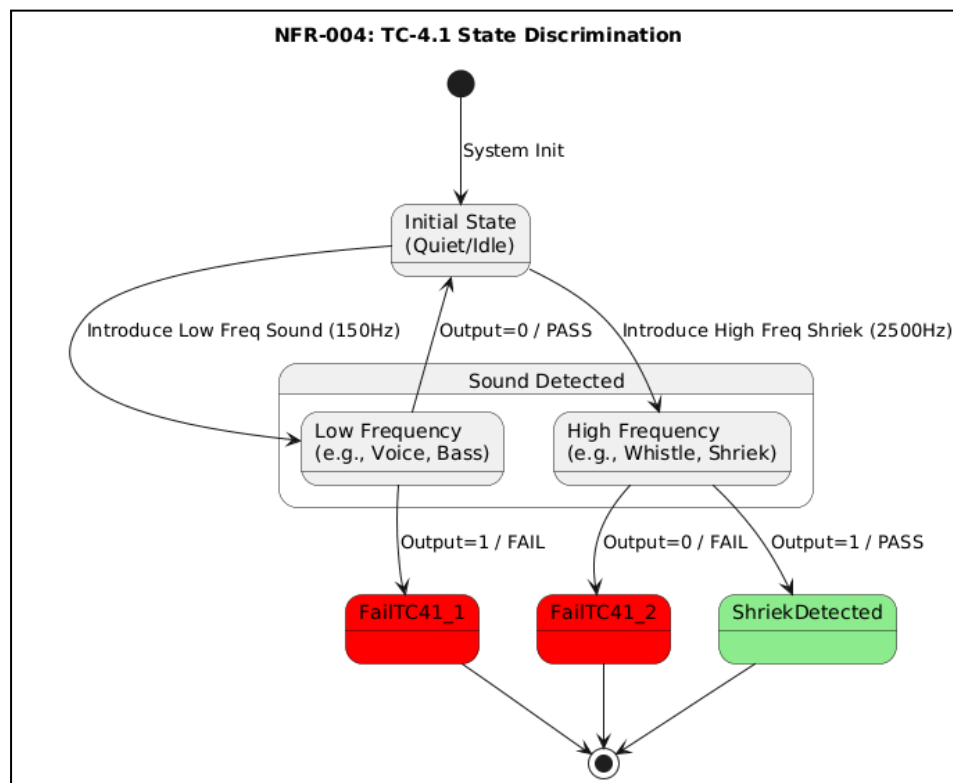


Figure 21. State Discrimination

Design Element ID D4.1.1

- Function: `read_microphone()` (Sampling & Preprocessing Phase)
- Description: Initiates and controls the high-speed sampling process to gather raw audio data, ensuring the data is correctly collected and prepared for frequency analysis.
- Key Actions:
 - Executes the sampling loop, calling `read_microphone()`
 - Enforces the fixed `SAMPLE_RATE_HZ` using a tight loop delay for sample precision.
 - Converts the collected raw `uint16_t` samples into float format for FFT processing.
 - Removes the DC bias by subtracting 2048 from each sample before FFT execution.
- Dependencies: Requires `adc_mic_init()` to be completed and relies on `read_microphone()` for data input.

Design Element ID: D4.2.1

- Function: `analyze_mic_fft()` (Discrimination & Output Phase)
- Description: Performs the Fourier analysis on the prepared sample buffer and executes the core logic for shriek classification (TC-4.1) based on frequency and magnitude thresholds.
- Key Actions:
 - Computes the FFT to transform time-domain audio data into the frequency spectrum.
 - Identifies the dominant frequency (`dominant_freq`) and its corresponding magnitude (`max_magnitude`) across the spectrum.
 - If a sound is detected but falls below the `SHRIEK_FREQ_THRESHOLD`, the logic implies the state is recognized as Low Frequency (e.g., Voice, Bass).
 - Compares the calculated peak frequency against `SHRIEK_FREQ_THRESHOLD` and the magnitude against `SHRIEK_MAG_THRESHOLD`.
 - Returns 1 (to transition to ShriekDetected / PASS) if both high frequency and high magnitude thresholds are exceeded; returns 0 otherwise.

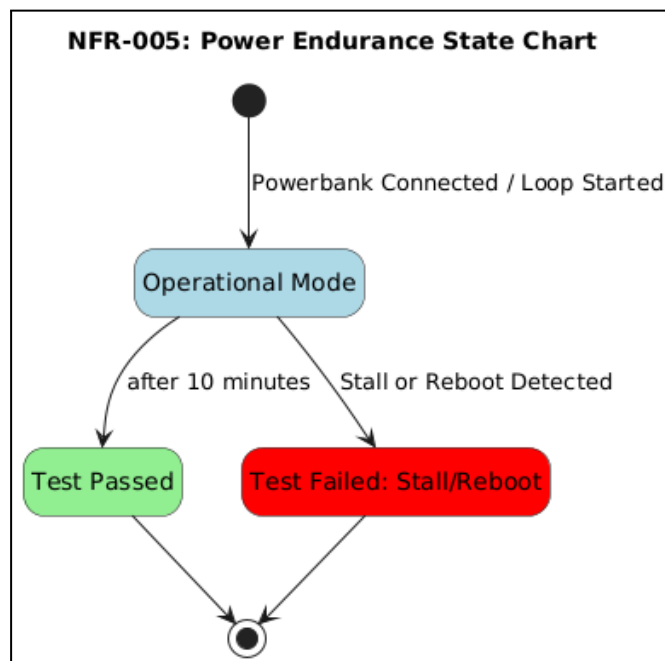


Figure 22. Power Endurance State Machine

- No design element for this state machine as it is a power source test not related to the system design.

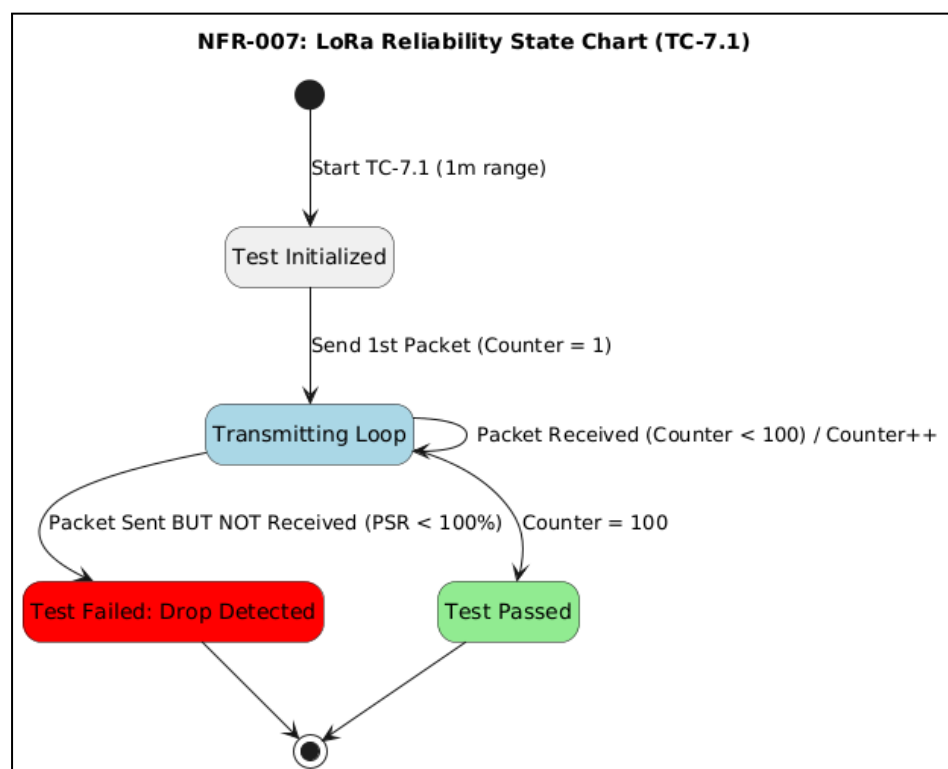


Figure 23. LoRa Reliability State Machine

Design Element ID: D5.1.

- **Function:** `lora_test()`
 - Calls required hardware initialization routines (e.g., `uart_init()` for the LoRa module).
 - Start Packet: Sends the 1st (sets the initial counter state to 1) to begin the test loop
 - Loop Entry: Maintains the Transmitting Loop state as long as the counter is less than 100.
 - Success Check: If `packet_status` is Received, increments the counter and immediately loops back to send the next packet.

- Failure Check: If `packet_status` is NOT Received:
 - Triggers the transition to the Test Failed: Drop Detected state.
 - Terminates the test early.
- Completion Check: When the counter = 1-00, checks the final status and triggers the transition to the Test Passed state.

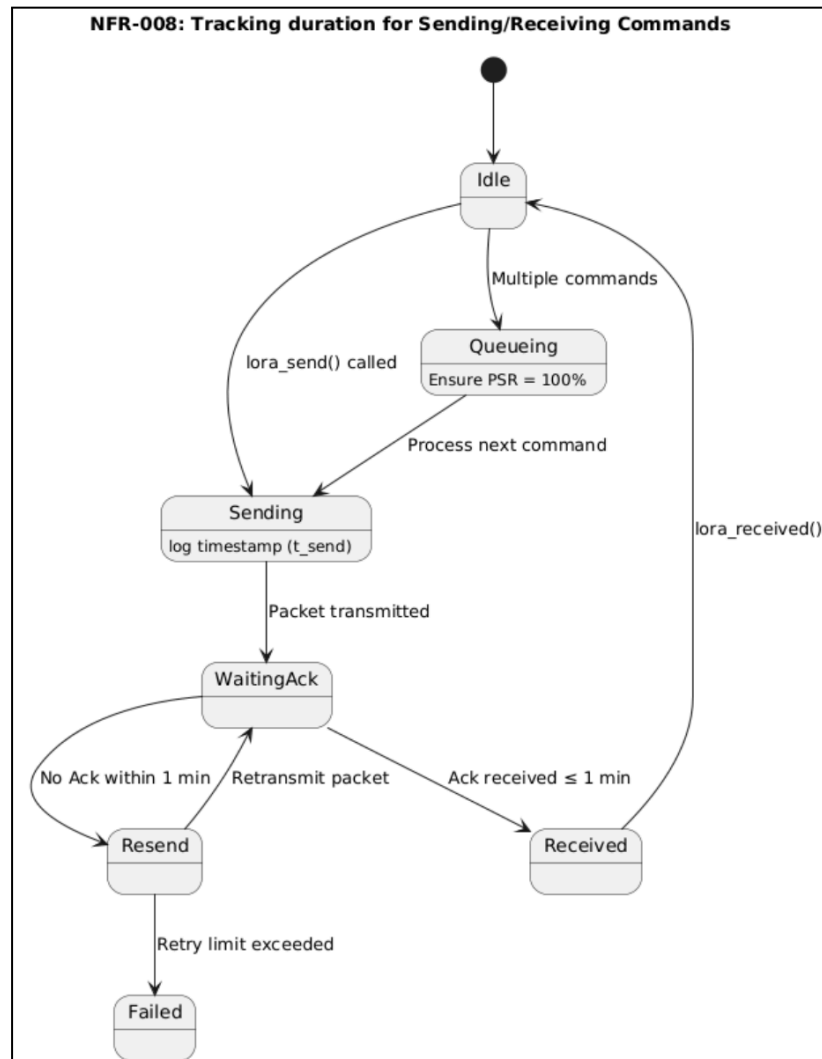


Figure 24. State machine for sending/receiving commands

Design Element ID: D4.2.1

- Function: `check_lora_messages()`
- Description:
 - Forwards commands from the Processor to the microcontroller to execute movements for the spider robot
 - Sends status feedback of the robot's current state from microcontroller to processor
- Key Actions:
 - Tracks duration of command sending using timestamps
 - If acknowledgement of command being sent from the end party is not confirmed, retransmit packet

Design Element ID: D4.2.2

- Function: `check_lora_messages()`
- Description: Checks for output from either Processor or Microcontroller and passing the packet across to the other party. The action is completed with an acknowledgement.

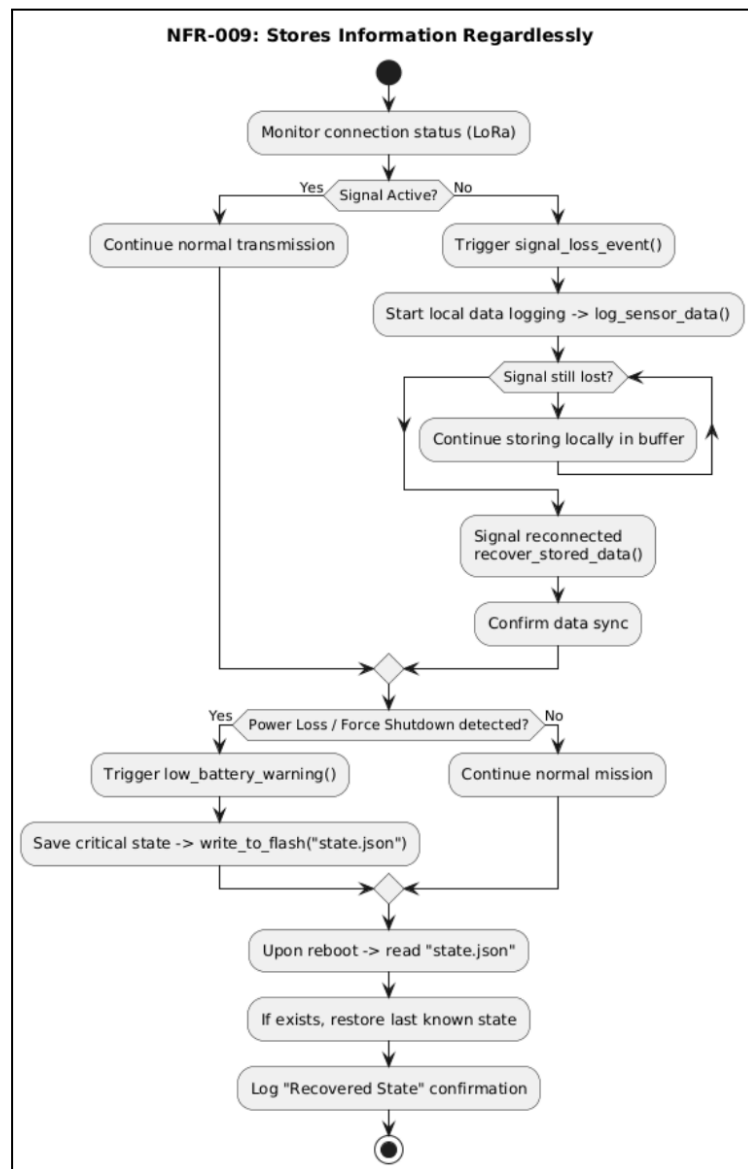


Figure 25. Flowchart on Data Persistence

Design Element ID: D4.3.1

- Function: To be implemented
- Description: If powerbank is low on battery, triggers a warning to activate critical saving state.
- Key Actions: Once action is called, all current data will be written into the microcontroller's flash memory using a json file to be recorded for safe keeping. When power resumes, recorded data is uploaded back from the last known state.

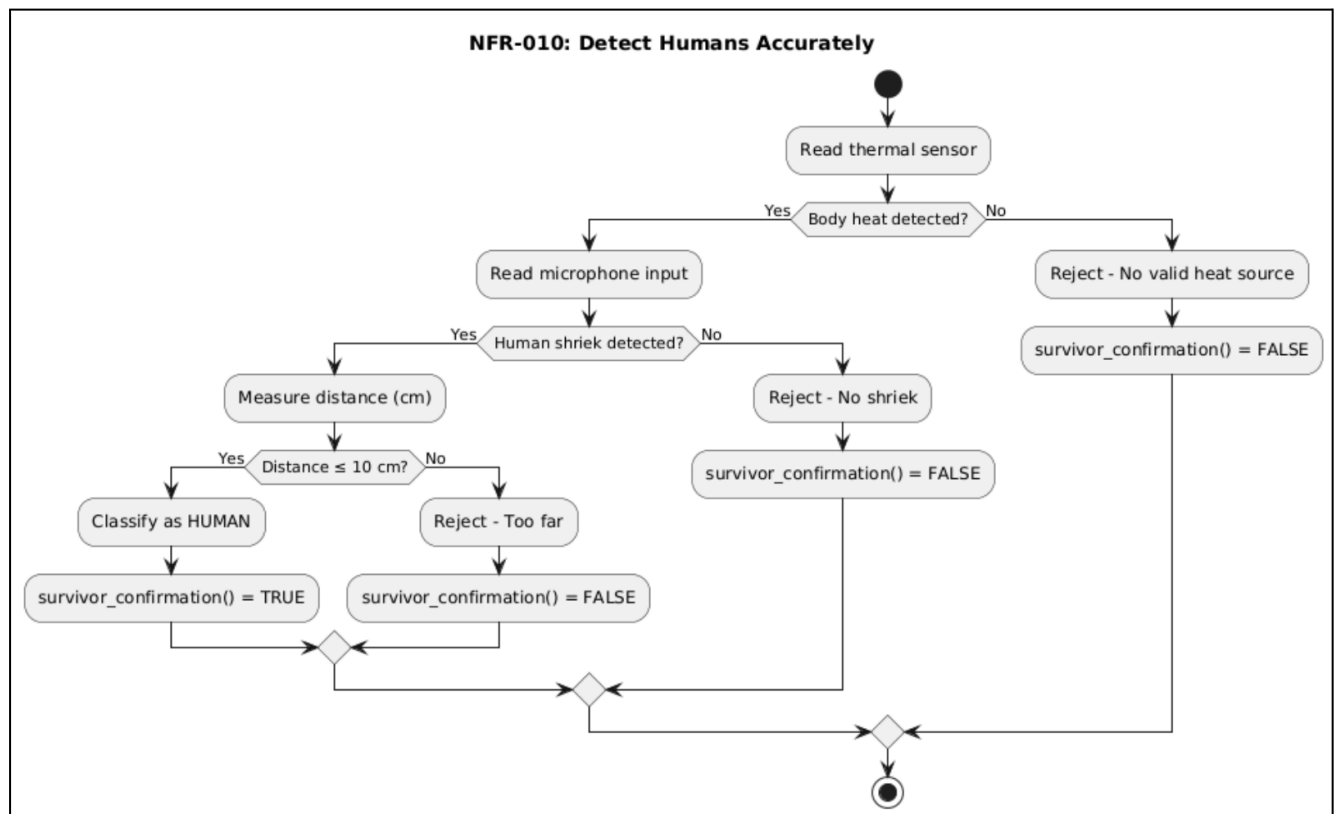


Figure 26. Flowchart on requirements to detect humans

Design Element ID: D3.2.5

- Function: survivor_confirmation()
- Description: Checks if the object detected is human or not. Sends an alert if data from the different sensors line up to be potentially human so to have more accurate finding if the object found is human or not.
- Key Actions: Once action is called, it will check through the different sensors. It first checks for the heat source in the surrounding area. If detected, it then checks if there is a human shriek being detected by the microphone sensor. If detected, it will then check if the object is within the range distance. If so, it will set the survivor_confirmation to true. If any of the checks is false, it will return false.

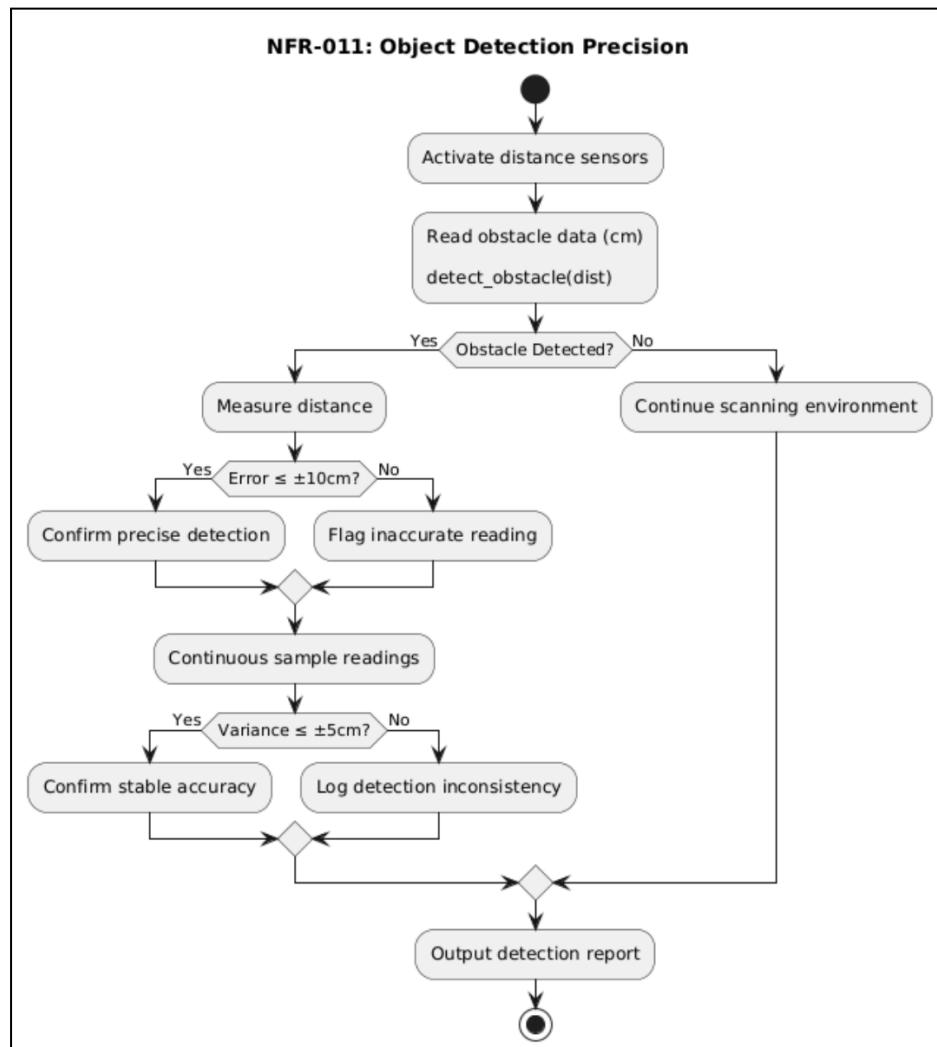


Figure 27. Object Detection Precision

Design Element ID: D3.1.6

- Function: detect_obstacle(dist)
- Description: Scans the surrounding for objects that come within its radius.
- Key Actions: The MM wave radar checks for objects in its surroundings, once an object is detected, the distance is recorded and tracked.

8. Test Plan & Results

Functional Requirement (FR-001): Standby Pose

- **TC-F1.1: Verify spider robot able to default to standby pose**
 - **Method:**
 - Power on the spider robot
 - **Pass Criteria:**
 - Spider robot legs will default to be in standby pose angle
 - Spider robot posture remains stable

Functional Requirement (FR-002): Basic Movement

- **TC-F2.1: Verify spider robot able to move forward**
 - **Method:**
 - Send SPIDER_MOVE_FORWARD command state to the spider
 - **Pass Criteria:**
 - Spider servo rotates to correct angles to form a move forward animation
 - Spider robot able to move forward from its current position

- **TC-F2.2: Verify spider robot able to move backwards**
 - **Method:**
 - Send SPIDER_MOVE_BACKWARD command state to the spider
 - **Pass Criteria:**
 - Spider servo rotates to correct angles to form a move backward animation
 - Spider robot able to move backward from its current position
- **TC-F2.3: Verify spider robot able to move left**
 - **Method:**
 - Send SPIDER_MOVE_LEFT command state to the spider
 - **Pass Criteria:**
 - Spider servo rotates to correct angles to form a move left animation
 - Spider robot able to move left from its current position
- **TC-F2.4: Verify spider robot able to move right**
 - **Method:**
 - Send SPIDER_MOVE_RIGHT command state to the spider
 - **Pass Criteria:**
 - Spider servo rotates to correct angles to form a move right animation
 - Spider robot able to move right from its current position
- **TC-F2.5: Verify spider robot able to stop after detecting an object/obstacle**
 - **Method:**
 - Send SPIDER_MOVE_FORWARD command state to the spider
 - Ensure there is an object blocking the path to the spider
 - **Pass Criteria:**
 - Spider robot is able to walk forward
 - Upon detecting the object, Spider robot is able to stop and default to Standby state
- **TC-F2.6: Verify spider robot able to perform custom walk**
 - **Method:**
 - Send CUSTOM_WALK command state to the spider
 - **Pass Criteria:**
 - Spider servo rotates to correct angles to form a move right animation
 - Spider robot able to move forward from its current position using the CUSTOM_WALK animation
- **TC-F2.7: Verify spider robot able to perform temperature walk**
 - **Method:**
 - Send TEMPERATURE_WALK command state to the spider
 - **Pass Criteria:**
 - Spider servo rotates to correct angles to form a move right animation
 - Spider robot able to move forward from its current position using the TEMPERATURE_WALK animation
- **TC-F2.8: Verify spider robot able to perform backtrack**
 - **Method:**
 - Send BACKTRACK command state to the spider
 - **Pass Criteria:**
 - Spider servo rotates to correct angles to form a move right animation
 - Spider robot able to rotate left using the SPIDER_ROTATE_LEFT animation
 - Spider robot able to rotate left once more using the SPIDER_ROTATE_LEFT animation
 - Spider robot able to move forward using the SPIDER_MOVE_FORWARD animation

- Spider is able to move forward in the opposition direction as its original direction

Functional Requirement (FR-003): Basic Turning

- **TC-F3.1: Verify spider robot able to rotate left**
 - **Method:**
 - Send SPIDER_ROTATE_LEFT command state to the spider
 - **Pass Criteria:**
 - Spider servo rotates to correct angles to form a rotate left animation
 - Spider robot able to rotate counter-clockwise from its current position
- **TC-F3.2: Verify spider robot able to rotate right**
 - **Method:**
 - Send SPIDER_ROTATE_Right command state to the spider
 - **Pass Criteria:**
 - Spider servo rotates to correct angles to form a rotate right animation
 - Spider robot able to rotate clockwise from its current position

Functional Requirement (FR-004): Heat Detection

- **TC-F4.1: Verify able to detect heat**
 - **Method:**
 - Monitor the communication link between the Sensor Pico and the Computer Pico
 - Human near the Heat Sensor
 - Observe the data being displayed in the Computer Pico through lora transmission
 - Human detected
 - Observe the data being displayed in the Computer Pico
 - **Pass Criteria:**
 - The Computer Pico successfully receives temperature data values from the Sensor Pico over the Lora Link
 - The reading with the heat source is at least 30°C higher
- **TC-F4.2: Verify able to recognise human temperature**
 - **Method:**
 - Monitor the communication link between the Sensor Pico and the Computer Pico
 - **High-Boundary Test:** Place a non-human heat source such as hot water (around 50°C) near the Heat Sensor
 - Observe the Sensor Pico and the Computer Pico
 - **Low-Boundary Test:** Replace the human heat source with a non-human one such as a room temperature water (around 25°C) near the Heat Sensor
 - Observe the Sensor Pico and Computer Pico once again
 - **Human Detection Test:** Replace the non-human heat source with a human one such as a human hand near the Heat Sensor
 - Observe the Sensor Pico and Computer Pico once again
 - **Pass Criteria:**
 - The Sensor Pico must transmit the Human Flag when human hand is present (around 36°C)
 - The Sensor Pico must NOT transmit the Human Flag for the 50°C high-boundary source

- The Sensor Pico must NOT transmit the Human Flag for the 10°C low-boundary source

Functional Requirement (FR-005): Sound Detection

- TC-F5.1: Human Frequency Detection :
 - **Method:** Play a 1500Hz tone at 70dB from a speaker positioned 1 meters from the microphone.
 - **Pass Criteria:** System must detect and log the frequency as within human range (300-3000Hz), trigger `analyze_mic_fft()`, and send alert via LoRa within **1 minute** of sound playback.
- TC-F5.2: Frequency Range Boundary Testing :
 - **Method:** Play tones at 300Hz, 250Hz, 3000Hz, and 3100Hz sequentially at 70dB.
 - **Pass Criteria:** System must detect **only** 300Hz and 3000Hz as valid human frequencies. 250Hz and 3100Hz must be ignored with no alert sent.
- TC-F5.3: Maximum Detection Range :
 - **Method:** Position speaker at 5 meters distance and play 1500Hz tone at 70dB.
 - **Pass Criteria:** System must detect the sound and send alert via LoRa. Repeat at 6 meters and verify **no detection** occurs (exceeds 5m range).
- TC-F5.4: Background Noise Rejection:
 - **Method:** Expose robot to ambient noise (40-50dB) for 30 seconds, then play 1500Hz tone at 70dB.
 - **Pass Criteria:** System must produce **zero false positives** during ambient noise period and successfully detect the 1500Hz tone with magnitude > **300 threshold**.

Functional Requirement (FR-006): Object Detection

- TC-F6.1: Basic Obstacle Detection :
 - **Method:** Place a solid obstacle (30cm × 30cm cardboard) at 100cm directly in front of the mmWave radar.
 - **Pass Criteria:** Radar must detect obstacles, calculate distance as **100cm ±10cm**, classify object type, and send detection data via LoRa within **200ms**.
- TC-F6.2: Minimum and Maximum Range Detection :
 - **Method:** Place obstacles at 15cm, 50cm, 80cm, and 100cm sequentially.
 - **Pass Criteria:** System must detect at 50cm and 80cm . Must **ignore** 15cm (blind zone < 25cm). Must detect at 100cm but **not detect** beyond 100cm.
- TC-F6.3: Human Classification via Movement :
 - **Method:** Have a person walk toward the radar from 3 meters at normal walking speed (~50cm/s).
 - **Pass Criteria:** Radar must classify object as OBJECT_HUMAN based on speed measurement > **15cm/s** and send [RADAR-HUMAN] message via LoRa.
- TC-F6.4: Stationary Object Classification :
 - **Method:** Place a stationary box at 0.8 meter for 5 seconds.
 - **Pass Criteria:** Radar must classify as MMWAVE_OBJECT_STATIONARY with **speed < 5cm/s** and **distance_stability < 10cm**.

Functional Requirement (FR-007): Multi-Sensor Confirmation

- TC-F7.1: Heat + Sound Confirmation :
 - **Method:** Position heat source at 37°C within 0.5m radius and play 1500Hz tone within 5m range simultaneously.
 - **Pass Criteria:** Function `survivor_confirmation()` must return **true**, system must log both sensor detections, and send confirmation alert via LoRa within **5 seconds**.
- TC-F7.2: Heat + Radar Human Confirmation :
 - **Method:** Position heat source at 37°C within 0.5m and have person move within radar range classified as MMWAVE_OBJECT_HUMAN.

- **Pass Criteria:** System must confirm survivor using heat (36-40°C) + radar human classification, and send [SURVIVOR CONFIRMED] alert via LoRa.
- TC-F7.3: Sound + Radar Human Confirmation :
 - **Method:** Play 1500Hz tone within 5m and have person move within radar range classified as OBJECT_HUMAN.
 - **Pass Criteria:** System must confirm survivor using sound (300-3000Hz) + radar human classification, and send alert via LoRa.
- TC-F7.4: Single Sensor False Positive Rejection :
 - **Method:** Trigger only heat sensor (37°C at 0.4m) with no sound or radar human detection for 10 seconds.
 - **Pass Criteria:** Temperature reading logged locally but no LoRa "human confirmed" message sent. request_temp_send flag remains **FALSE**.
- TC-F7.5: Out-of-Range Temperature Rejection :
 - **Method:** Position heat source at 50°C within 0.5m and play 1500Hz tone within 5m.
 - **Pass Criteria:** Temperature reading 50°C logged locally. [MIC ALERT] sent via LoRa but NO "human confirmed" message sent despite sound detection.

Functional Requirement (FR-008): Path Routing to Survivors

- TC-F8.1: Navigate to Heat Source (Clear Path) :
 - **Method:** Position heat source at 37°C exactly 2 meters ahead with no obstacles in path.
 - **Pass Criteria:** When heat source (37°C) is detected and confirmed as HUMAN_CONFIRMED, system must execute COMMAND_TEMPERATURE_WALK, maintain forward heading via COMMAND_MOVE_WALK, and stop when **distance < 50cm (COMMAND_MOVE_HALT triggered)** with LoRa confirmation sent within **30 seconds**.
- TC-F8.2: Obstacle Avoidance During Navigation :
 - **Method:** Position heat source 3m ahead and place obstacle at 1m in direct path.
 - **Pass Criteria:** When obstacle detected at **≤100cm via radar (WALL/MMWAVE_OBJECT_STATIONARY status)**, system must execute COMMAND_MOVE_HALT within **200ms**.
- TC-F8.3: Multi-Sensor Navigation Priority :
 - **Method:** Position heat source at 2m ahead and play 1500Hz sound from the same direction. Place off-center obstacle at 1.5m.
 - **Pass Criteria:** System must confirm survivor (heat + sound), detect obstacle via radar, calculate adjusted path avoiding obstacle, and navigate to survivor within **< 50cm**.
- TC-F8.4: Heat Signature Verification During Movement :
 - **Method:** Position heat source 10cm away, initiate navigation, then remove heat source after robot moves 5cm.
 - **Pass Criteria:** Robot must initiate COMMAND_TEMPERATURE_WALK when heat detected (37°C at 10cm). If heat reading drops below 36°C (out of range), system must execute COMMAND_MOVE_HALT, stop movement, and not send "survivor reached" confirmation. Must return to exploration state via DFS algorithm.

Functional Requirement (FR-009): Navigate and Pathfind

TC-9.1: Dead End Backtracking

- TC-F9.1.a: Dead End Recognition
 - **Method:** Place the robot 30cm from a wall in a short, closed corridor.
 - **Pass Criteria:** The robot must **detect the wall and immediately stop** within 1 second. The system must confirm the current location is a **dead end** after a 360 degree scan.
- TC-F9.1.b: Map Logging
 - **Method:** Immediately check the robot's internal memory/map structure after TC-F9.1.a.

- **Pass Criteria:** The coordinates of the dead end must be **logged** and marked as **'explored/blocked'** to prevent future visits.
- TC-F9.1.c: Backtrack Maneuver
 - **Method:** Allow the robot to initiate the exit maneuver from the dead end.
 - **Pass Criteria:** Robot must execute a precise **180 degree $\pm$ 5 degree turn**. The turn must complete within $\geq$ 3 seconds.
- TC-F9.1.d: Return to Fork
 - **Method:** Allow the robot to continue moving until it reaches the previously known fork (Waypoint).
 - **Pass Criteria:** The robot must successfully return to the previous fork taking the same time as it did to reach the dead end, confirming successful retreat.

TC-9.2: Path Selection at a Fork

- TC-F9.2.a: Path Scanning
 - **Method:** Place the robot at a three-way fork (Left, Forward, Right) with all paths currently marked as 'unexplored'.
 - **Pass Criteria:** Robot must execute a **360 degree scan** and log three distinct open paths (vectors) into its decision queue.
- TC-F9.2.b: Selection Logic
 - **Method:** Block the Left and Forward paths (simulating obstacles), leaving only the Right path open.
 - **Pass Criteria:** The Path Finding Algorithm must calculate and select the **Right Path** as the next destination.
- TC-F9.2.c: Turn Execution
 - **Method:** Allow the robot to initiate the turn toward the Right Path.
 - **Pass Criteria:** Robot must execute a precise **90 degree $\pm$ 5 degree turn** and immediately initiate the **Move Forward** command.

Non-Functional Requirement 1 (NFR-001): Continuous Operation (Servo Health)

- TC-N1.1: Servo Overload Test
 - **Method:** Run the robot to execute continuous commands, and the minimum requirement for it to be activated in standby state for 5 mins without overheating
 - **Pass Criteria:** The system must be able to execute commands sent and hold standby state without overheating for a minimum of 5 mins

Non-Functional Requirement 2 (NFR-002): Sensor Failure Handling

- TC-N2.1: MM Wave Radar Failure Response
 - **Method:** While moving, force the radar's read_and_send_radar function to **return an error code consistently** for longer than **500ms**.
 - **Pass Criteria:** When radar data is unavailable for $>500\text{ms}$ (mmwave_process_data returns false consistently), system must log "[ERROR] Radar failure detected", execute COMMAND_MOVE_HALT to stop movement, transition to error state, and send LoRa alert "[RADAR_FAILURE]". System must continue navigation using temperature and sound sensors only.
- TC-N2.2: Microphone Failure Response
 - **Method:** During a search, force the read_microphone function to **return zero data** (simulated failure) for 500ms.
 - **Pass Criteria:** When radar data is unavailable for $>500\text{ms}$ (mmwave_process_data returns false consistently), system must log "[ERROR] Radar failure detected", execute COMMAND_MOVE_HALT to stop movement, transition to error state, and send LoRa alert "[RADAR_FAILURE]". System must continue navigation using temperature and sound sensors only.
- TC-N2.3: Heat Sensor Failure Response

- **Method:** During a search, force the read_and_send_temperatures function to **return zero data** (simulated failure) for 500ms.
- **Pass Criteria:** When read_and_send_temperatures() returns 0.0°C consistently for >500ms (out of biological range), system must log "[ERROR] Temperature sensor failure detected", send LoRa alert "[TEMP_FAILURE]", and continue human detection using radar (MMWAVE_OBJECT_HUMAN type) and sound (1500Hz distress detection) without temperature verification requirement.

Non-Functional Requirement 3 (NFR-003): Quick Obstacle Response

- TC-N3.1: Quick Stop Response Time
 - **Method:** Move robot toward a wall, triggering detection at 30cm and above.
 - **Pass Criteria:** When an obstacle is detected at between 30cm to 100cm distance (speed < 1 cm/s, MMWAVE_OBJECT_STATIONARY type), the system must execute COMMAND_CRIT_STANDBY within one sensor cycle. Verify stop by checking that speed_cm_s remains < 1 cm/s and distance_stability ≤ 10cm in subsequent detections. Response time from detection frame to halt command execution ≤ 1 second. Log detection with frame number and timestamp via LoRa.
- TC-N3.2: Custom Walk
 - **Method:** After the object stops, the CUSTOM_WALK command is sent to the spider if object distance ≥ 0.66m.
 - **Pass Criteria:** Object should move forward (closer to the object) for the number of cycles set in spider command. It should complete all cycles and stop in front of the object.
- TC-N3.3: Strafing Decision Accuracy
 - **Method:** Place an obstacle directly ahead and a clear path 15cm to the right.
 - **Pass Criteria:** When obstacle detected as MMWAVE_OBJECT_STATIONARY with distance_stability < 10cm (confirming solid object) and at center heading, system must execute COMMAND_MOVE_STRAFE. After strafe, verify obstacle distance increases or moves off-center (heading angle changes). Resume forward movement with COMMAND_MOVE_WALK only after confirming clear path (distance > 100cm or detection dropped). Log strafing decision and post-strafe detection count confirmation via LoRa showing successful obstacle avoidance.

Non-Functional Requirement 4 (NFR-004): Detect and Report Distress Sounds Promptly

- TC-N4.1: Accurate Shriek and Noise Discrimination
 - **Method:** Execute read_microphone function. Introduce low frequency sounds (e.g. a deep voice) and high frequency sounds (e.g. whistle or shriek).
 - **Pass Criteria:** The low-frequency sound must return 0, indicating non-shriek. The high-frequency shriek must return 1, indicating shriek detected.
- TC-N4.2: Mic Status Transmission and Integrity
 - **Method:** Manually force the read_microphone function to transmit data and observe LoRa terminal on the PC.
 - **Pass Criteria:** The received data string on the PC must contain 1 or 0 in the Mic Frequency Field, confirming the Pico transmits the status correctly.

Non-Functional Requirement 5 (NFR-005): Operate on Power Bank for a Minimum Period

- TC-N5.1: Power Source Reliability
 - Method: Power the robot and Picos using the powerbank
 - Pass Criteria: The robot and both Picos must remain operational and run the sensor reading/motor control loop without rebooting or stalling for a minimum of 10 minutes

Non-Functional Requirement 6 (NFR-006): Power Bank Easy to Replace

- TC-N6.1: Power Source Replacement Speed
 - Method: Use a stopwatch to time the process of replacing the power bank.
 - Pass Criteria: The physical system must be built in a way where the power bank is accessible to ensure this, the total replacement time (disconnect to reconnect) must be less than 30 seconds.

Non-Functional Requirement 7 (NFR-007): Reliable LoRa Communication

- TC-N7.1: Packet Reliability
 - Method: Position Pico #1 (Sender) and the PC LoRa module (Receiver) 1 meter apart. Send 100 consecutive data packets via `check_lora_messages()`. Count the number of packets successfully received by the PC.
 - Pass Criteria: The Packet Success Rate (PSR) must be 95%.

Non-Functional Requirement 8 (NFR-008): Low Delay in Sending/Receiving Commands

- TC-N8.1: Command Transmission Delay
 - Method: Send a command packet over LoRa while logging timestamps from `check_lora_messages()` and acknowledgement of the functions.
 - Pass Criteria: Round-trip acknowledgement time ≤ 1 minute
- TC-N8.2: Command Queue Handling
 - Method: Send 10 commands in rapid succession (interval ≤ 1 minute)
 - Pass Criteria: No command dropped, Packet Success Rate (PSR) must be 100%

Non-Functional Requirement 9 (NFR-009): Stores information Regardlessly

- TC-N9.1: Signal Loss Logging
 - Method: Simulate LoRa disconnection for 10 minutes
 - Pass Criteria: continues logging locally without loss
 - Pass Criteria: On reconnection, recovers stored data uploads successfully
- TC-N9.2: Critical State Flash Storage
 - Method: During mission execution, cut power to simulate force shutdown, after a triggered a low battery warning event.
 - Pass Criteria: 'state.json' contains the last saved flash written data (mode, position)
 - Pass Criteria: On reboot, the Pico reads the json file and logs 'Recovered state'

Non-Functional Requirement 10 (NFR-010): Detect Humans Accurately

- TC-N10.1: Human Body Detection
 - Method: Human stand within 5-10 cm of distance from the heat sensor and shriek
 - Pass Criteria: `SendHumanFoundCommand()` flag correct as human
- TC-N10.2: False Positive Rejection
 - Method: Place heat source (eg, lamp or hot surface) in front of the sensor
 - Pass Criteria: System must not classify detected heat as human

Non-Functional Requirement 11 (NFR-011): Object Detection Precision

- TC-N11.1: Distance Accuracy
 - Method: Place small/flat obstacles at known distance (0.50m , 0.80m, 1m)

- Pass Criteria: Report distance error its less than 1m
- TC-N11.2: Consistency Under Motion
 - Method: Move robot laterally 10cm past a fixed obstacle, logging readings
 - Pass Criteria: Reading variance $\leq \pm 5\text{cm}$ from continuous sampling.

9. Traceability (Week 9 & Week 13)

Req ID	Requirement	Design Ref	Code Ref	Test Ref	Pass/Fail	
FR-001	The spider robot shall be able to default to its standby pose upon powering on	HLD Figure 2, HLD Figure 4, Figure 11, Blocks D2.1.1 D2.1.2	main.c: servo_leg_state(SPIDER_STANDBY)	TC-F1.1	Pass	
FR-002	The spider robot shall be able to move in forward, backward, left and right direction using its multi-legged servo system	HLD Figure 2, HLD Figure 4, HLD Figure 5, Figure 11, Blocks D2.1.1 D2.1.2	main.c: servo_leg_state(SPIDER_MOVE_FORWARD)	TC-F2.1 TC-F2.5	TC-2.1	Pass
					TC-2.5	Fail
FR-002		HLD Figure 2, HLD Figure 4, Figure 11, Blocks D2.1.1 D2.1.2	main.c: servo_leg_state(SPIDER_MOVE_BACKWARD)	TC-F2.2	Pass	
FR-002		HLD Figure 2, HLD Figure 4, Figure 11, Blocks D2.1.1 D2.1.2	main.c: servo_leg_state(SPIDER_MOVE_LEFT)	TC-F2.3	Pass	
FR-002		HLD Figure 2, HLD Figure 4, Figure 11, Blocks D2.1.1 D2.1.2	main.c: servo_leg_state(SPIDER_MOVE_RIGHT)	TC-F2.4	Pass	
FR-002		HLD Figure 2, HLD Figure 4, Figure 11, Blocks D2.1.1 D2.1.2	main.c: servo_leg_state(SPIDER_CUSTOM_WALK)	TC-F2.6	Pass	

		HLD Figure 2, HLD Figure 4, Figure 11, Blocks D2.1.1 D2.1.2	main.c: servo_leg_state(SPIDER_TEMPE RATURE_WALK)	TC-F2.7	Pass	
FR-002		HLD Figure 2, HLD Figure 4, Figure 11, Blocks D2.1.1 D2.1.2	main.c: servo_leg_state(SPIDER_BACKT RACK)	TC-F2.8	Pass	
FR-003	The spider robot shall be able to rotate clockwise and anti-clockwise direction using its multi-legged servo system	HLD Figure 2, HLD Figure 4, Figure 11, Blocks D2.1.1 D2.1.2	main.c: servo_leg_state(SPIDER_ROTAT E_LEFT)	TC-F3.1	Pass	
FR-003		HLD Figure 2, HLD Figure 4, Figure 11, Blocks D2.1.1 D2.1.2	main.c: servo_leg_state(SPIDER_ROTAT E_RIGHT)	TC-F3.2	Pass	
FR-004	The spider robot shall be able to detect human heat signature (between 36°C to 40°C) using the heat sensor within 10 cm radius ±5cm (This is the maximum range of the sensor)	HLD Figure 7, Figure 16, Blocks D7.2.2 D7.2.3 D7.2.4 D7.4.1	main.c: read_and_send_t emperatures	TC-F4.1 TC-F4.2	TC-4.1	Pass
					TC-4.2	Pass
FR-005	The spider robot shall be able to capture and analyze audio frequencies between 300-3000Hz to detect human distress calls using the microphone within 1 meter radius ±10cm.	HLD Figure 8, Figure 15, Blocks D8.2.2-8.2.5	main.c: read_microphone	TC-F5.1 TC-F5.2 TC-F5.3 TC-F5.4	TC-5.1	Pass
					TC-5.2	Pass

	(This is the maximum range of the sensor)				TC-5.3	Pass
					TC-5.4	Pass
FR-006	The spider robot shall be able to detect nearby objects using the mm wave radar within 0.2 meters radius. Threshold is from 30cm to 100cm	HLD Figure 9, Figure 13 Blocks D9.2.1-9.2.5	main.c: read_and_send_radar	TC-F6.1 TC-6.2 TC-6.3 TC-6.4	TC-6.1	Fail
					TC-6.2	Pass
					TC-6.3	Pass
					TC-6.4	Pass
FR-007	The spider robot shall be able to confirm nearby survivors using its heat and sound detection within 5 meter radius. (This is the maximum range of the sensor)	HLD Figure 3, Figure 12, Block D7.2.1,D8.2.1, D7.2.5	main.c: survivor_confirmation	TC-F7.1 TC-F7.2 TC-F7.3 TC-F7.4 TC-F7.5	TC-7.1	Fail
					TC-7.2	Pass
					TC-7.3	Pass
					TC-7.4	Pass
					TC-7.5	Pass
FR-008	The spider robot shall be able to navigate to potential survivors by combining Heat, Sound and Object Detection	HLD Figure 3, Figure 14, Blocks D3.6.1-3.6.4	main.c: survivor_confirmation	TC-F8.1 TC-F8.2 TC-F8.3 TC-F8.4	TC-8.1	Pass
					TC-8.2	Pass
					TC-8.3	Fail
					TC-8.4	Fail
FR-009	The spider robot shall be able to navigate to different areas to find potential survivors by combining the Object Detection and	HLD Fig 3, Figure 17, Blocks D3.1.1, D3.3.1, D3.4.1-D3.4.2, D3.5.1	algorithm.c: DFS_decision_maker(), Fork_scan_paths() control.c Control_execute_action(),	TC-F9.1 TC-F9.2	TC-9.1a	Pass
					TC-9.1b	Pass
					TC-9.1c	Fail
					TC-9.1d	Pass
					TC-9.2a	Pass

	its Path Finding Algorithm				TC-9.2b	Pass
					TC-9.2c	Fail
NFR-001	Robot should operate continuously without the servo dying	HLD Fig 1	-	TC-N1.1	Pass	
NFR-002	System should continue to function if sensor fails	HLD Fig 9, HLD Fig 8, HLD Fig 7 Figure 19, Blocks D9.3.1, D8.3.2, D7.3.2,	main.c: sensor_fail_handler	TC-N2.1 TC-N2.2 TC-N2.3	TC-N2.1	Fail
					TC-N2.2	Fail
					TC-N2.3	Fail
NFR-003	Detect and respond to nearby obstacle quickly	HLD Fig 3, Figure 20, Blocks D2.8.1, D3.4.3	algorithm.c: DFS_decision_maker(), Fork_scan_paths() control.c Control_execute_action(),	TC-N3.1 TC-N3.2 TC-N3.3	TC-N3.1	Fail
					TC-N3.2	Pass
					TC-N3.3	Fail
NFR-004	Detect and report distress sounds promptly	HLD Figure 8, Figure 4, Blocks D4.1, D4.2	main.c: read_microphone	TC-N4.1 TC-N4.2	TC-N4.1	Pass
					TC-N4.2	Pass
NFR-005	Operate on power bank for a minimum period	-	-	TC-N5.1	Pass	
NFR-006	Power bank easy to replace	-	-	TC-N6.1	Pass	

NFR-007	Reliable long-range (Lora) communication	HLD Figure 9, 8, 7 Blocks D5.1	main.c: check_lora_messages	TC-N7.1	Pass	
NFR-008	Low delay in sending and receiving commands	HLD Figure 4, Figure 24, Blocks D4.2.1,D4.2.2	main.c: check_lora_messages()	TC-N8.1, TC-N8.2	TC-N8.1	Pass
					TC-N8.2	Fail
NFR-009	Store information even if signal is lost	HLD Figure 4, Figure 25 Blocks D4.3.1	-	TC,-N9.1 TC-N9.2	TC-N9.1	Fail
					TC-N9.2	Fail
NFR-010	Detect humans accurately	HLD Figure 3, Blocks, Figure 26 D3.2.5	main.c: SendHumanFoundCommand()	TC-N10.1, TC-N10.2	TC-N10.1	Pass
					TC-N10.2	Pass
NFR-011	Obstacle detection precision	HLD Figure 3, Blocks, Figure 27 D3.1.6	main.c: detect_obstacle()	TC-N11.1, TC-N11.2	TC-N11.1	Pass
					TC-N11.2	Fail

11. Sign-off

Adli	Build Spider, Servos
Swathi	Build Spider, Algorithm
Subhashini	MMWave Radar, LoRa sensor
Srushti	Temperature Sensor, Microphone Sensor
Shyann	Build Spider, Spider Communication

References:

【MicroPython】machine.UART Function - Waveshare Wiki. (2025). Waveshare.com.

https://www.waveshare.com/wiki/%E3%80%90MicroPython%E3%80%91machine.UART_Function

Grove - Analog Microphone | Seeed Studio Wiki. (2022, January 6). Seeedstudio.com.

<https://wiki.seeedstudio.com/Grove-Analog-Microphone/>

Shenzhen Hi-Link Electronic Co., Ltd. (2023). *HLK-LD2450 motion target detection and tracking module instruction manual* [PDF]. TinyTronics.

https://www.tinytronics.nl/product_files/006000_HLK-LD2450-Instruction-Manual.pdf

Instructables. (2024, July 8). *How to Connect MLX90614 Infrared to Raspberry Pi Pico*. Instructables.

<https://www.instructables.com/How-to-Connect-MLX90614-Infrared-to-Raspberry-Pi-P/>

mcauser. (2016, September 24). *GitHub - mcauser/micropython-mlx90614: MicroPython driver for the MLX90614 IR temperature sensor*. GitHub. <https://github.com/mcauser/micropython-mlx90614>

Melexis MLX90614 IR Temperature Sensor - I2C (App Note). (2025). Labjack.com.

<https://support.labjack.com/docs/melexis-mlx90614-ir-temperature-sensor-i2c-app-not>

Spider Robotic Kits for Arduino Programming Professional ESP8266 Automation Robot Complete Educational Learning Electronic Kit - AliExpress 502. (2025). Aliexpress.

<https://www.aliexpress.com/item/1005006281636034.html>

Welcome To Zscaler Directory Authentication. (2025). Espressif.com.

https://docs.espressif.com/projects/esp-at/en/release-v2.2.0.0_esp8266/AT_Command_Set/TCPIP_AT_Commands.html

